KalSea: Kalman-Corrected Spectral-Evolution-Aware Caching

Supplementary Material

This supplement gives implementation details (Sec. A), the derivation of the gap smoother (Sec. B), the evaluation protocol, including the comparison with published results (Sec. C), additional results on FLUX.1-dev, including more qualitative examples (Sec. D), results on the development set (Sec. E), approaches that did not help (Sec. F), the video experiments: protocol, gate correction, budgets, same-setup comparisons, ablations and budget sweep (Sec. G), and cost measurements (Sec. H). Sections, tables, figures and equations of this document are numbered A, B, ... and S1, S2, ...; plain numbers refer to the main paper.

A. Implementation details

A.1. Kalman residual forecaster

Forecast quantity and coordinates. The forecaster tracks the block-stack residual of the image tokens, $\mathbf{r}_i = \mathbf{h}_i^{\text{out}} - \mathbf{h}_i^{\text{in}}$, which for FLUX.1-dev at 1024^2 has 4096 tokens on a 64×64 grid and 3072 channels. In the DCT variant, every channel is transformed with the orthonormal 2D DCT-II $\mathcal{D}$ on the token grid, and the coefficient with frequency indices (u, v) belongs to band ℓ if its normalized radius $r = \sqrt{(u/64)^2 + (v/64)^2}/\sqrt{2}$ lies in $[e_\ell, e_{\ell+1})$, with edges $\{0, \frac{1}{64}, \frac{1}{32}, \frac{1}{16}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1\}$ (seven bands, Tab. S1). The coefficients of band ℓ are divided by ς_ℓ, where ς_ℓ^2 is the calibrated mean power of the residual's coefficients in that band, so that the filter operates in relative units; this scaling cancels in the correction (Sec. B). The per-element variant omits the DCT and uses a single band whose scale is derived from the mean of the seven calibrated powers.

Time coordinate. Following SpecKCast [10], every band has its own time coordinate derived from the noise schedule. With the log signal-to-noise ratio $\lambda_i = 2\log((1-\sigma_i)/\sigma_i)$ (σ_i clamped to $[10^{-4}, 1-10^{-4}]$) and $t_\ell(i) = \text{sigmoid}\big((\lambda_i + \log p_\ell)/T_w\big)$ with $T_w = 2$,

$$s_\ell(i) = 2\,\frac{t_\ell(i) - t_\ell(0)}{t_\ell(N-1) - t_\ell(0)} - 1 \tag{S1}$$

increases from -1 to 1 over the $N = 50$ steps, and $\Delta s_{\ell,i} = s_\ell(i) - s_\ell(i-1)$. Here p_ℓ is the calibrated power of the clean-image estimate in band ℓ: the clock of a band advances fastest when the signal-to-noise ratio of its frequencies is close to one. The per-element variant uses the count-weighted mean of the p_ℓ.

Filter. Each coefficient (DCT variant) or element (per-element variant) has the state $\mathbf{s} = [m, \nu]^\top$, and all coefficients of a band share the covariance $\mathbf{P}_\ell$, hence the Kalman and smoother gains. Prediction and update follow (4) and (5) of the main paper with $\Delta s = \Delta s_{\ell,i}$, $\tau = 0.5$ and $R = 10^{-3}$. The filter predicts at every step and is updated only at computed steps. On skipped steps the cache writes the predicted values, transformed back to residual units: $\hat{\mathbf{r}}_i = \mathcal{D}^{-1}(\mathbf{m}_i^- \odot \boldsymbol{\varsigma})$, where the map $\boldsymbol{\varsigma}$ holds ς_ℓ on the coefficients of band ℓ.

Table S1. **FLUX.1-dev calibration of the DCT forecaster** (16 uncached trajectories, COCO captions disjoint from all evaluation prompts). Band ℓ: radial range of the normalized DCT frequency r on the 64×64 token grid; n_ℓ: coefficients per channel; ς_ℓ^2: mean power of the residual's DCT coefficients (normalization); p_ℓ: power of the clean-image estimate (time warp); q_ℓ: process-noise intensity (relative units); the initial trend variance is $2q_\ell$.

ℓ	r	n_ℓ	ς_ℓ^2	p_ℓ	q_ℓ
0	[0, 1/64)	3	1.53×10^7	2.02×10^3	2.28×10^3
1	[1/64, 1/32)	5	2.94×10^5	496	1.37×10^3
2	[1/32, 1/16)	22	8.23×10^4	147	616
3	[1/16, 1/8)	82	2.18×10^4	25.2	459
4	[1/8, 1/4)	312	7.55×10^3	5.22	632
5	[1/4, 1/2)	1230	3.48×10^3	0.915	2.69×10^3
6	[1/2, 1)	2442	2.56×10^3	0.237	1.10×10^4

Initialization. At the first computed step (step 0, which the gate always computes), $m = z_0$, $\nu = 0$, $\mathbf{P}_\ell = \text{diag}(R,\, 2q_\ell)$ and $\kappa_\ell = 1$, where q_ℓ is the calibrated process-noise intensity.

Process-noise adaptation. At a computed step b, before the update, the filter compares the mean squared innovation over all coefficients and channels of band ℓ, $\overline{e^2_{\ell,b}}$, with its predicted variance $S_{\ell,b} = [\mathbf{P}^-_{\ell,b}]_{00} + R$:

$$\begin{aligned} \gamma_{\ell,b} &= \text{clip}_{[0.1,10]}\big(\overline{e^2_{\ell,b}}/S_{\ell,b}\big), \\ \kappa_\ell &\leftarrow \text{clip}_{[0.1,10]}\big(\kappa_\ell\, \gamma_{\ell,b}^{0.2}\big). \end{aligned} \tag{S2}$$

The update at b uses the gain computed before this change; the new κ_ℓ scales $\mathbf{Q}$ from the next prediction on [9].

Constants. $\tau = 0.5$, $R = 10^{-3}$, $T_w = 2$, the exponent 0.2 and the clipping ranges in (S2) are the same for all budgets and models. They were fixed on the development set; $\tau = 0.1$ is the default of SpecKCast [10] (ablated in Tab. S6, group (d)).

A.2. Calibration

The DCT variant is calibrated once for FLUX.1-dev from 16 uncached 50-step trajectories at 1024^2 with guidance 3.5, one per COCO caption [6]; the 16 captions are disjoint from the 64 development and the 200 DrawBench prompts. For a fixed random subset of 384 of the 3072 channels we record the block-stack residual at every step, together with the latent input and the velocity output of the transformer. From these,

- ς_ℓ^2 is the mean squared DCT coefficient of the residual in band ℓ, over steps, channels and trajectories;
- p_ℓ is the mean power in band ℓ of the clean-image estimate $\hat{\mathbf{x}}_0 = \mathbf{x}_i - \sigma_i \mathbf{v}_i$ over the last 25 steps (latent DCT coefficients, grouped by the same physical frequencies as the token-grid bands);
- q_ℓ is, per trajectory, the median over steps of the mean squared change of the finite-difference trend $\nu_i = (z_{i+1} - z_i)/\Delta s_{\ell,i+1}$ (relative units) divided by the mean of the two adjacent band-time steps, averaged over the trajectories;
- the initial trend variance is $2q_\ell$.

Tab. S1 lists the values. No evaluation prompt is used for calibration.

A.3. Per-element and calibration-free variants

The *per-element* variant uses the same calibration, reduced to one band (count-weighted means of p_ℓ, q_ℓ and the initial trend variance; scale: square root of the mean of the ς_ℓ^2). The *calibration-free* per-element variant needs no trajectories: $q = 7400$, initial trend variance 15000, a time coordinate that is uniform in the step index (linear from -1 to 1), and as scale the root mean square of the first observed residual. All other constants are those of Sec. A.1. On FLUX.1-dev, both variants perform like the DCT filter once PCGS is applied (Tab. S6, group (d), and Tab. S7); we use the calibration-free variant for video (Sec. G).

A.4. Predictor-corrector gap smoothing

Bookkeeping. For each skipped step i of the current gap, the cache keeps the latent input $\mathbf{x}_i$, the conditioning $\mathbf{c}_i$, the forecast $\hat{\mathbf{r}}_i$ and the prior covariances $\mathbf{P}^-_{\ell,i}$ of all bands. At the update of the closing computed step b it records $\mathbf{P}^-_{\ell,b}$ and the innovation. The gains $g_{i,\ell}$ of (6) are computed by the backward recursion $\mathbf{\Phi}(b,i) = \mathbf{\Phi}(b,i{+}1)\,\mathbf{A}_{i+1}$ over the gap, from quantities the filter maintains anyway (illustrative values: Tab. S2).

Correction in residual space. For the DCT variant, $\tilde{\mathbf{r}}_i = \hat{\mathbf{r}}_i + \mathcal{D}^{-1}\big(\mathbf{G}_i \odot \mathcal{D}(\mathbf{r}_b - \hat{\mathbf{r}}_b)\big)$, where the gain map $\mathbf{G}_i$ has the value $g_{i,\ell}$ on all coefficients of band ℓ (derivation in Sec. B); for the per-element variant $\mathbf{G}_i$ is the scalar g_i.

Re-integration, continuation and gate refresh. The gap is replayed from the latent that the sampler used at the first skipped step, $\mathbf{x}_{a+1}$, with (8): Emb (the patch embedding) and Out (final adaptive normalization and linear projection) are evaluated once per skipped step, and each Euler step uses the arithmetic of the diffusers flow-matching scheduler (the bf16 velocity times the fp32 step size is added to the fp32 copy of the bf16 latent, then cast to bf16). With the identity smoother ($\tilde{\mathbf{r}}_i = \hat{\mathbf{r}}_i$), the replay reproduces the uncorrected run of the per-element filter bitwise (unit test on the GPU). At step b, the returned velocity is (9), computed in fp32 and cast to bf16. The gate refresh recomputes the SEA gate's stored input of step b from $\mathbf{x}'_b$: one adaptive normalization of the first block on $\mathrm{Emb}(\mathbf{x}'_b)$ and one SEA filter. The last step is always computed; its gap is corrected in the same way, and no refresh is needed.

Tests. In float64 on synthetic residuals with the FLUX.1-dev calibration and a gate-like schedule (9 gaps, 36 skipped steps), the implemented gains agree with brute-force joint-Gaussian conditioning of the damped-trend model to a relative error of $8 \cdot 10^{-16}$ (DCT bands) and $6 \cdot 10^{-16}$ (per element). With every step computed there is no gap, and the images are bitwise equal to the 50-step reference.

NFE accounting and cost. NFE is counted by forward pre-hooks on all 19 double-stream and 38 single-stream blocks (block calls / 57) and cross-checked against the forward pass's own record of computed steps: every DrawBench run has zero mismatches and no partial block stack, and PCGS never calls a block. Per image, KalSea evaluates Emb and Out 44.0 times at 18.1 NFE and 47.0 times at 13 NFE (skipped steps plus corrected computed steps; Tab. S11), each about 3.4 GFLOP at 1024^2 (linear maps between 64 and 3072 channels over 4096 tokens; one transformer evaluation costs 59.5 TFLOPs, 2976 for 50 steps [2]), plus 11.2 and 9.0 gate refreshes, in total below 0.3% of one transformer evaluation. The peak allocated memory exceeds that of the uncached sampler by 0.16 GiB (KalSea w/o PCGS: 0.14 GiB; SeaCache: 0.05 GiB). Wall-clock times: Sec. H.

B. Derivation of the gap smoother

Setting. Consider one Kalman filter of the predictor (one residual element, or one DCT band whose coefficients share the covariance). Let $a < b$ be consecutive computed steps, $\mathcal{Y}_a$ the observations up to step a, and $a < i < b$ a skipped step. No observation falls into the gap, so the predicted moments are propagated without update: $\mathbf{s}^-_i = \mathbf{A}_i \mathbf{s}^-_{i-1}$ and $\mathbf{P}^-_i = \mathbf{A}_i \mathbf{P}^-_{i-1} \mathbf{A}_i^\top + \mathbf{Q}_i$ for $a+1 < i \le b$, starting from $\mathbf{s}^-_{a+1} = \mathbf{A}_{a+1}\mathbf{s}^+_a$. Define the transition $\mathbf{\Phi}(b,i) = \mathbf{A}_b \mathbf{A}_{b-1} \cdots \mathbf{A}_{i+1}$, with $\mathbf{\Phi}(b,b) = \mathbf{I}$, and the innovation variance $S_b = \mathbf{h}^\top \mathbf{P}^-_b \mathbf{h} + R$ with $\mathbf{h} = [1,0]^\top$.

Proposition. For $a < i \le b$,

$$\mathbb{E}[\mathbf{s}_i \mid \mathcal{Y}_a, z_b] = \mathbf{s}_i^- + \mathbf{P}_i^- \mathbf{\Phi}(b,i)^\top \mathbf{h}\, S_b^{-1} e_b, \quad e_b = z_b - \mathbf{h}^\top \mathbf{s}_b^-, \tag{S3}$$

and its first component gives the gain $g_i = [\mathbf{P}_i^- \mathbf{\Phi}(b,i)^\top]_{00}/S_b$ of (6).

Proof by conditioning. Iterating the state equation, $\mathbf{s}_b = \mathbf{\Phi}(b,i)\mathbf{s}_i + \mathbf{w}_{i\to b}$, where $\mathbf{w}_{i\to b}$ is a sum of process-noise terms of steps $i+1,\dots,b$; it has zero mean and is independent of $\mathbf{s}_i$ and of $\mathcal{Y}_a$. With $z_b = \mathbf{h}^\top \mathbf{s}_b + \eta_b$ and $\eta_b \sim \mathcal{N}(0,R)$ independent of everything else, $\mathrm{Cov}(\mathbf{s}_i, z_b \mid \mathcal{Y}_a) = \mathbf{P}_i^- \mathbf{\Phi}(b,i)^\top \mathbf{h}$ and $\mathrm{Var}(z_b \mid \mathcal{Y}_a) = S_b$. Conditioning the jointly Gaussian pair $(\mathbf{s}_i, z_b)$ given $\mathcal{Y}_a$ on z_b gives (S3).

Equivalence with the RTS smoother. The RTS recursion [11, 12] over $[a,b]$, using the observations up to b, reads $\mathbf{s}_i^{\mathrm{s}} = \mathbf{s}_i^+ + \mathbf{C}_i(\mathbf{s}_{i+1}^{\mathrm{s}} - \mathbf{s}_{i+1}^-)$ with $\mathbf{C}_i = \mathbf{P}_i^+ \mathbf{A}_{i+1}^\top (\mathbf{P}_{i+1}^-)^{-1}$, initialized with the filtered estimate $\mathbf{s}_b^{\mathrm{s}} = \mathbf{s}_b^+ = \mathbf{s}_b^- + \mathbf{P}_b^- \mathbf{h} S_b^{-1} e_b$. Inside the gap, $\mathbf{s}_i^+ = \mathbf{s}_i^-$ and $\mathbf{P}_i^+ = \mathbf{P}_i^-$. We show by backward induction that $\mathbf{s}_i^{\mathrm{s}} - \mathbf{s}_i^- = \mathbf{P}_i^- \mathbf{\Phi}(b,i)^\top \mathbf{h} S_b^{-1} e_b$. The case $i = b$ holds because $\mathbf{\Phi}(b,b) = \mathbf{I}$. If it holds for $i+1$, then

$$\begin{aligned} \mathbf{s}_i^{\mathrm{s}} - \mathbf{s}_i^- &= \mathbf{P}_i^- \mathbf{A}_{i+1}^\top (\mathbf{P}_{i+1}^-)^{-1} \mathbf{P}_{i+1}^- \mathbf{\Phi}(b,i+1)^\top \mathbf{h} S_b^{-1} e_b \\ &= \mathbf{P}_i^- \big(\mathbf{\Phi}(b,i+1)\mathbf{A}_{i+1}\big)^\top \mathbf{h} S_b^{-1} e_b, \end{aligned}$$

and $\mathbf{\Phi}(b,i+1)\mathbf{A}_{i+1} = \mathbf{\Phi}(b,i)$. □

Consequences.

- At the newest observation the smoothed and filtered estimates coincide, so PCGS leaves the predictor for the next gap unchanged (main paper, Sec. 3.3, Remarks).
- *Random walk.* For the model without trend ($\mathbf{A} = 1$, $\mathbf{Q} = q\Delta s$), an exact observation at a ($\mathbf{P}_a^+ = 0$) and $R \to 0$, the gain is $g_i = (s_i - s_a)/(s_b - s_a)$, *i.e.*, linear interpolation in the time coordinate. With the reuse forecast $\hat{\mathbf{r}}_i = \mathbf{r}_a$ this is the linear interpolation between $\mathbf{r}_a$ and $\mathbf{r}_b$ used for reuse with PCGS in the ablations.
- *DCT variant.* Each band ℓ has its own $\mathbf{A}$, $\mathbf{Q}$ and $\mathbf{P}$, hence its own gains $g_{i,\ell}$. The band normalization multiplies z_b, m_b^- and m_i^- by the same factor and cancels in (S3), which gives the residual-space correction of Sec. A.4.
- *Gain shape.* Tab. S2 shows the gains for the gap 33–41 of the most common 13-NFE schedule with κ=1: they rise from 0.04–0.11 at the first skipped step to 0.80–0.87 at the last one, and the highest-frequency bands rise most slowly.

Why smoothing minimizes the latent error to first order. Write $h_i = \sigma_{i+1} - \sigma_i$ and let $\boldsymbol{\rho} = (\boldsymbol{\rho}_{a+1}, \dots, \boldsymbol{\rho}_{b-1})$ be the residuals used to replay the gap, $\mathbf{x}_{i+1} = \mathbf{x}_i + h_i\,\mathrm{Out}(\mathrm{Emb}(\mathbf{x}_i) + \boldsymbol{\rho}_i, \mathbf{c}_i)$, from the exact $\mathbf{x}_{a+1}$. Let $\mathbf{r} = (\mathbf{r}_{a+1}, \dots, \mathbf{r}_{b-1})$ be the residuals that full computation would produce and $\mathbf{x}_b^\star$ the corresponding latent. Linearizing the replay around the fully computed trajectory,

$$\mathbf{x}_b(\boldsymbol{\rho}) - \mathbf{x}_b^\star \approx \sum_{i=a+1}^{b-1} h_i\, \mathbf{M}_{b,i}\, \mathbf{J}_i\, (\boldsymbol{\rho}_i - \mathbf{r}_i) = \mathbf{L}\,(\boldsymbol{\rho} - \mathbf{r}), \tag{S4}$$

where $\mathbf{J}_i$ is the Jacobian of Out with respect to its first argument and $\mathbf{M}_{b,i} = \prod_{j=i+1}^{b-1}(\mathbf{I} + h_j \mathbf{J}_j \mathbf{E}_j)$ propagates a latent perturbation from step $i+1$ to b ($\mathbf{E}_j$: Jacobian of Emb, a linear map for FLUX.1-dev). For any fixed linear map $\mathbf{L}$ and any estimate $\boldsymbol{\rho}$ that depends only on the observations $\mathcal{Y}$, $\mathbb{E}[\|\mathbf{L}(\boldsymbol{\rho} - \mathbf{r})\|^2 \mid \mathcal{Y}] = \|\mathbf{L}(\boldsymbol{\rho} - \mathbb{E}[\mathbf{r} \mid \mathcal{Y}])\|^2 + \mathbb{E}[\|\mathbf{L}(\mathbf{r} - \mathbb{E}[\mathbf{r} \mid \mathcal{Y}])\|^2 \mid \mathcal{Y}]$, which is minimal for $\boldsymbol{\rho} = \mathbb{E}[\mathbf{r} \mid \mathcal{Y}]$. Under the filter's model, the conditional means given the observations up to b are the smoothed residuals $\tilde{\mathbf{r}}_i$, whereas the forecasts $\hat{\mathbf{r}}_i$ are the conditional means given the observations up to a. Hence, to first order and under the filter's model, reintegrating with $\tilde{\mathbf{r}}_i$ minimizes the expected squared error of $\mathbf{x}_b'$ among all estimates available at step b. The argument covers the DCT variant because the DCT is linear and the conditional mean is taken per coefficient. It does not cover model misspecification, which is why simpler gains can do as well in practice (main paper, Sec. 4.4).

Table S2. **RTS gains $g_{i,\ell}$ of the DCT forecaster** for the gap 33–41 of the most common 13-NFE schedule (0-based steps), per band ℓ (0: lowest frequencies), from the FLUX.1-dev calibration with the process-noise scale fixed to κ=1 (no online adaptation, so the gains do not depend on the image). The gains rise across the gap; early in the gap they are smaller for the high-frequency bands.

step i	ℓ=0	ℓ=1	ℓ=2	ℓ=3	ℓ=4	ℓ=5	ℓ=6
34	0.11	0.10	0.10	0.09	0.07	0.05	0.04
35	0.23	0.22	0.21	0.19	0.17	0.13	0.10
36	0.35	0.35	0.34	0.32	0.28	0.23	0.19
37	0.48	0.48	0.47	0.45	0.41	0.35	0.30
38	0.62	0.61	0.61	0.59	0.56	0.50	0.44
39	0.74	0.74	0.74	0.73	0.71	0.66	0.61
40	0.87	0.87	0.87	0.87	0.86	0.83	0.80

C. Evaluation protocol

Published results and the mapping from δ to NFE. SeaCache reports compute as TFLOPs (2976 for its 50-step original, *i.e.*, 59.5 per transformer evaluation). We convert its published rows to NFE (Tab. S3); these are the rows marked † in the main paper's Tab. 1. Its larger-budget row is labelled $\delta = 0.3$, but its 1098 TFLOPs imply 18.45 evaluations. With the released gate, $\delta = 0.3$ computes 20.88 steps per image on average in our setup (20 or 21 per prompt), and $\delta = 0.37$ computes 18.14; its $\delta = 0.6$ row (12.99) agrees with our measurement (13.00, on every prompt). We cannot tell whether the TFLOPs or the threshold of the $\delta = 0.3$ row are off. Running our cache at the NFE implied by the

Table S3. **Published FLUX.1-dev results of SeaCache [2] (its Table 1) and their NFE.** NFE $= 50 \times$ TFLOPs$/2976$, where 2976 TFLOPs is their 50-step original; for Δ-DiT and ToCa, which compute parts of the network, it is a FLOP equivalent. Bottom: our re-run of SeaCache (released gate, reuse) at three thresholds, in our setup (DrawBench-200, B200, seed = prompt index).

Method (published)	TFLOPs	NFE	PSNR↑	SSIM↑	LPIPS↓
Original (50 steps)	2976	50.00	–	–	–
Vanilla, 25 steps‡	1487	24.98	15.553	0.668	0.409
TeaCache [7] (δ=0.3)	1547	25.99	20.762	0.810	0.211
TaylorSeer [8] (S=3)	1191	20.01	22.783	0.828	0.163
SeaCache [2] (δ=0.3)	1098	18.45	26.285	0.893	0.106
Δ-DiT [1]	1984	33.33	17.403	0.710	0.336
ToCa [16]	1263	21.22	18.398	0.700	0.324
Vanilla, 15 steps‡	892	14.99	17.842	0.740	0.305
TeaCache [7] (δ=0.6)	892	14.99	17.214	0.714	0.348
TaylorSeer [8] (S=5)	834	14.01	19.972	0.762	0.236
SeaCache [2] (δ=0.6)	773	12.99	21.332	0.798	0.226
SeaCache, our re-run					
δ=0.3	–	20.88	26.89	0.904	0.086
δ=0.37	–	18.14	25.33	0.880	0.113
δ=0.6	–	13.00	21.26	0.800	0.214

‡As published; the 25-step row scores below the 15-step row on every metric, so the two rows appear swapped.

TFLOPs ($\delta = 0.37$) is the conservative choice in both cases: if their run used 18.4 evaluations, the comparison is matched; if it used 20.9, SeaCache had more compute than our cache. Our re-run of SeaCache reaches 26.89 dB at $\delta = 0.3$, close to the published 26.29 dB, and 25.33 dB at $\delta = 0.37$; at $\delta = 0.6$ it reproduces the published 21.33 dB (21.26 dB). The published numbers were obtained on different hardware (NVIDIA Blackwell Pro 6000, according to the paper) and with seeds the paper does not state, so the main paper compares with them without confidence intervals, and with our re-run with paired intervals (Tab. S5). The published 25- and 15-step rows appear swapped (the 25-step row is worse on every metric); we cite them as published.

Matched schedules. Within one threshold, all our caches use the same SEA gate, but the gate is closed loop: its decisions depend on the latent and hence on the filler. Tab. S4 counts the prompts on which two or three caches computed exactly the same set of steps. At the thresholds of the main comparisons, SeaCache, KalSea w/o PCGS and KalSea computed the same steps on 191, 188 and 182 of the 200 prompts ($\delta = 0.37$, 0.3 and 0.6), any two of them on 183 to 196, and the per-prompt NFE never differs by more than 1. All ablation runs of Tab. S6 are within 0.06 NFE of KalSea at $\delta = 0.3$ and at exactly 13.00 NFE at $\delta = 0.6$.

Metrics. Images are compared with the uncached 50-step output of the same code path, prompt and seed (seed = DrawBench prompt index, 200 prompts in the file order). PSNR: scikit-image on 8-bit RGB with data range 255. SSIM: scikit-image structural similarity on 8-bit RGB (default 7×7 uniform window, channels averaged). LPIPS: AlexNet features [14], inputs scaled to $[-1, 1]$. SeaCache does not state its SSIM implementation or LPIPS backbone (TeaCache's evaluation code, on which it builds, uses a Gaussian 11×11 SSIM window), so published SSIM and LPIPS values may not be exactly comparable with ours; PSNR on 8-bit RGB is unambiguous up to the data range. Confidence intervals: paired percentile bootstrap over prompts, 10,000 resamples, with the same resampling indices for every pair with the same number of prompts.

Table S4. **Matched schedules.** Number of the 200 DrawBench prompts on which the caches computed exactly the same set of steps (same SEA gate and δ; the gate is closed loop, so the filler can change its decisions): all three, or the pair KalSea/SeaCache (O/S), KalSea/KalSea w/o PCGS (O/K), KalSea w/o PCGS/SeaCache (K/S); and the largest per-prompt difference in NFE between any two of them.

δ	NFE	all three	O/S	O/K	K/S	max $\lvert\Delta\rvert$ NFE
0.2	27.00	199	200	199	199	0
0.3	20.84	188	192	196	188	1
0.37	18.14	191	195	195	192	1
0.5	15.00	172	180	185	178	0
0.6	13.00	182	188	193	183	0
0.8	11.00	147	172	165	153	0
1.0	9.00	130	155	167	132	0

Reproducibility. The DCT filter accumulates its band statistics with atomic additions on the GPU (scatter-add), so two runs of the same configuration are not bitwise identical; with deterministic algorithms enforced they are. On DrawBench, a second run of KalSea differs by +0.02 dB [−0.04, 0.08] at $\delta = 0.3$ and +0.04 dB [−0.003, 0.10] at $\delta = 0.6$ (Tab. S6, caption), and two independent runs of KalSea w/o PCGS differ by +0.03 dB [−0.003, +0.07] at $\delta = 0.37$ and +0.002 dB [−0.01, +0.01] at $\delta = 0.6$, with identical computed steps on 200/200 and 200/200 prompts (the main paper's Tab. 1 uses one of these runs, the 2×2 ablation of Tab. S7 the other). On the development set, a same-session re-run of KalSea w/o PCGS differs by −0.05 and +0.03 dB (Tab. S14). We therefore do not interpret differences below 0.1 dB. The per-element filter, reuse and the uncached reference regenerated bit-identical images on all 64 development prompts; for video, the filter updates run with deterministic algorithms, and re-generated videos are bit-identical (Sec. G.7).

D. Additional results on FLUX.1-dev

Same-setup comparisons at three thresholds. Tab. S5 lists all same-setup comparisons with SeaCache, linear ex-

Table S5. **Same-setup comparisons on FLUX.1-dev** (DrawBench-200; same code, seeds and hardware; all methods share the SEA gate and δ). Δ: paired difference KalSea minus method, mean and 95% bootstrap CI over prompts; win: fraction of prompts with higher PSNR for KalSea. δ=0.37 is the budget of the main paper's Tab. 1, δ=0.3 the threshold of SeaCache's larger budget (linear extrapolation was run at δ=0.3 and 0.6 only).

Method	NFE	PSNR↑	SSIM↑	LPIPS↓	ΔPSNR (dB)	ΔSSIM	ΔLPIPS	win
δ=0.3 *(20.8 NFE)*								
SeaCache (re-run)	20.88	26.89	0.904	0.086	+2.73 [+2.34, +3.12]	+0.031 [+0.027, +0.036]	−0.033 [−0.037, −0.029]	88.0%
Linear extrapolation	20.84	28.51	0.925	0.061	+1.11 [+0.89, +1.34]	+0.010 [+0.008, +0.012]	−0.009 [−0.010, −0.007]	83.0%
KalSea w/o PCGS	20.82	28.79	0.927	0.060	+0.83 [+0.63, +1.02]	+0.008 [+0.006, +0.009]	−0.007 [−0.009, −0.006]	81.0%
KalSea	20.84	29.62	0.935	0.053		(reference of Δ)		
δ=0.37 *(18.1 NFE)*								
SeaCache (re-run)	18.14	25.33	0.880	0.113	+2.39 [+2.03, +2.75]	+0.036 [+0.032, +0.040]	−0.043 [−0.047, −0.038]	87.5%
KalSea w/o PCGS	18.13	26.91	0.907	0.080	+0.81 [+0.63, +1.00]	+0.009 [+0.008, +0.011]	−0.010 [−0.012, −0.008]	80.5%
KalSea	18.14	27.72	0.916	0.070		(reference of Δ)		
δ=0.6 *(13.0 NFE)*								
SeaCache (re-run)	13.00	21.26	0.800	0.214	+1.27 [+1.02, +1.52]	+0.042 [+0.035, +0.048]	−0.060 [−0.068, −0.052]	79.0%
Linear extrapolation	13.00	21.80	0.827	0.171	+0.73 [+0.54, +0.92]	+0.015 [+0.011, +0.018]	−0.017 [−0.022, −0.012]	75.5%
KalSea w/o PCGS	13.00	21.91	0.826	0.172	+0.61 [+0.48, +0.76]	+0.016 [+0.013, +0.019]	−0.018 [−0.021, −0.015]	78.0%
KalSea	13.00	22.53	0.842	0.154		(reference of Δ)		

trapolation and KalSea w/o PCGS, with paired intervals for PSNR, SSIM and LPIPS. At $\delta = 0.3$ (20.8 NFE), the threshold of SeaCache's larger-budget row, KalSea reaches 29.62 dB, 2.73 dB [2.34, 3.12] above our re-run of SeaCache (26.89 dB at 20.88 NFE) and 0.83 dB [0.63, 1.02] above KalSea w/o PCGS (28.79 dB). At 18.1 NFE, the median per-prompt gain of PCGS over KalSea w/o PCGS is 0.57 dB, and PCGS lowers PSNR on 39 of the 200 prompts, by at most 4.75 dB; at 13 NFE the corresponding values are 0.44 dB, 44 prompts and 2.31 dB.

Ablations. Tab. S6 extends the main paper's Tab. 3 with all metrics, the continuation variants (group (c)), the forecasters without correction (group (d)) and the forecasting targets (group (f)). Re-embedding the corrected latents matters at the larger budget: the batched re-integration on the stored inputs $\mathbf{h}_i^{\text{in}}$ loses 0.22 dB [0.09, 0.34] at 20.8 NFE and 0.06 dB [−0.01, 0.13] at 13 NFE. The shape of the gains matters little: a linear interpolation from $\mathbf{r}_a$ to $\mathbf{r}_b$ and the forecast plus a linearly weighted innovation match the RTS gains at 13 NFE and exceed them by 0.14 dB [0.04, 0.25] at 20.8 NFE (group (b) of Tab. 3); we keep the RTS gains, which were chosen on the development set. Tab. S7 crosses the two forecaster parametrizations with PCGS at the budgets of Tab. 1.

PCGS on other predictors. Tab. S8 isolates the gain of PCGS for each predictor at matched NFE. It improves reuse by 1.69 dB [1.51, 1.88] and 0.74 dB [0.62, 0.86], and linear extrapolation by 1.15 dB [0.92, 1.38] and 0.80 dB [0.62, 0.98], at 20.8 and 13 NFE, more than the Kalman forecasters. With PCGS, linear extrapolation ties KalSea, while reuse stays below it (Tab. S6, group (e)), so a forecasting predictor remains useful.

Budget sweep. Tab. S9 is the table view of the main paper's Fig. 3: seven thresholds from 27 to 9 NFE, with the paired gains of KalSea over our re-run of SeaCache and over KalSea w/o PCGS. Tab. S10 adds the forecasts without correction. As the open-loop argument predicts, the gain of the Kalman forecast over reuse falls from +2.37 dB [2.02, 2.71] at 27 NFE to +0.11 dB [−0.14, 0.34] at 11 NFE and −0.10 dB [−0.31, 0.10] at 9 NFE, and linear extrapolation falls below reuse, by 0.58 and 0.77 dB at 11 and 9 NFE, as forecast errors accumulate over long gaps.

Size of the correction. Tab. S11 reports the relative latent correction $\|\mathbf{x}'_b - \mathbf{x}_b\|/\|\mathbf{x}_b\|$ that PCGS applies at the computed step closing each gap. It averages 0.70% at 20.8 NFE, 1.00% at 18.1 NFE and 2.00% at 13 NFE, and it grows with the length and lateness of the gap: on the most common 13-NFE schedule, from 0.2% after the single skipped steps early in sampling to 5–6% after the last two long gaps.

Open-loop forecast error inside gaps. Tab. S12 quantifies the drift that motivates PCGS (main paper, Sec. 1). Along six uncached development trajectories, a forecaster that observes only the steps of the 13-NFE schedule produces velocities whose relative error grows from 4–5% at the first skipped step of a seven-step gap to more than 20% (21.6–26.8%) at the seventh for the residual forecasts of KalSea w/o PCGS.

Qualitative results. Figs. S1 and S2 show the prompts at the lower quartile, the median and the upper quartile of the per-prompt PSNR gain of KalSea over SeaCache (our re-run of its released gate with reuse) at 13 and 20.8 NFE, selected by a fixed rule. Tab. S13 lists the prompts and seeds of the main paper's Fig. 1.

Variant	$\delta = 0.3$ (KalSea: 20.84 NFE) NFE	PSNR↑	ΔPSNR [95% CI]	SSIM↑	LPIPS↓	$\delta = 0.6$ (KalSea: 13.00 NFE) NFE	PSNR↑	ΔPSNR [95% CI]	SSIM↑	LPIPS↓
KalSea: Kalman forecast + PCGS	20.84	29.62	0 (ref.)	0.935	0.053	13.00	22.53	0 (ref.)	0.842	0.154
(a) Skip-path filler at the same gate, no correction										
Reuse (SeaCache)	20.88	26.89	−2.73 [−3.12, −2.34]	0.904	0.086	13.00	21.26	−1.27 [−1.52, −1.02]	0.800	0.214
First-order linear extrapolation	20.84	28.51	−1.11 [−1.34, −0.89]	0.925	0.061	13.00	21.80	−0.73 [−0.92, −0.54]	0.827	0.171
Kalman forecast (KalSea w/o PCGS)	20.82	28.79	−0.83 [−1.02, −0.63]	0.927	0.060	13.00	21.91	−0.61 [−0.76, −0.48]	0.826	0.172
(b) Gap correction on the Kalman forecast (KalSea: RTS gains)										
Forecaster state only, no re-integration	20.82	28.75	−0.86 [−1.06, −0.68]	0.927	0.061	13.00	21.89	−0.63 [−0.78, −0.50]	0.823	0.174
Re-integrate: linear interp. $\mathbf{r}_a \to \mathbf{r}_b$	20.84	29.76	+0.14 [+0.04, +0.25]	0.936	0.052	13.00	22.53	+0.00 [−0.05, +0.06]	0.839	0.156
Re-integrate: quadratic interp.	20.84	29.57	−0.05 [−0.16, +0.06]	0.935	0.054	13.00	22.45	−0.08 [−0.16, −0.00]	0.840	0.156
Re-integrate: pred. + linear innovation	20.84	29.76	+0.14 [+0.04, +0.25]	0.936	0.052	13.00	22.55	+0.02 [−0.03, +0.07]	0.840	0.155
(c) Re-integration and continuation										
Batched (zeroth-order) re-integration	20.84	29.40	−0.22 [−0.34, −0.09]	0.933	0.054	13.00	22.46	−0.06 [−0.13, +0.01]	0.840	0.156
No gate-state refresh	20.90	29.65	+0.03 [−0.04, +0.10]	0.936	0.052	13.00	22.54	+0.01 [−0.01, +0.03]	0.842	0.154
Block output held fixed at b	20.84	29.65	+0.03 [−0.03, +0.10]	0.935	0.053	13.00	22.54	+0.01 [−0.02, +0.04]	0.842	0.154
+ forecaster re-referenced	20.84	29.61	−0.01 [−0.13, +0.13]	0.935	0.053	13.00	22.54	+0.01 [−0.02, +0.04]	0.842	0.154
(d) Forecaster parametrisation										
Per-element filter, no correction	20.84	28.79	−0.82 [−1.03, −0.63]	0.927	0.060	13.00	21.94	−0.59 [−0.79, −0.40]	0.829	0.169
Per-element filter + PCGS	20.84	29.63	+0.01 [−0.15, +0.14]	0.935	0.053	13.00	22.52	−0.01 [−0.15, +0.13]	0.843	0.153
τ=0.1, no correction	20.84	28.45	−1.17 [−1.40, −0.95]	0.923	0.066	13.00	21.74	−0.79 [−0.97, −0.60]	0.817	0.186
τ=0.1 + PCGS	20.85	29.54	−0.08 [−0.29, +0.12]	0.934	0.054	13.00	22.58	+0.05 [−0.11, +0.22]	0.840	0.156
Calibration-free per-element, no corr.	20.84	28.75	−0.87 [−1.07, −0.67]	0.927	0.060	13.00	22.00	−0.53 [−0.73, −0.33]	0.830	0.169
Calibration-free per-element + PCGS	20.84	29.60	−0.02 [−0.11, +0.08]	0.936	0.052	13.00	22.55	+0.02 [−0.13, +0.17]	0.843	0.153
(e) Correction on other fillers										
Reuse + PCGS (linear smoother)	20.88	28.58	−1.03 [−1.39, −0.67]	0.927	0.061	13.00	22.00	−0.53 [−0.76, −0.29]	0.827	0.173
Linear extrapolation + PCGS	20.84	29.66	+0.04 [−0.08, +0.16]	0.935	0.052	13.00	22.59	+0.07 [−0.05, +0.18]	0.844	0.152
(f) Forecasting target, no correction (DCT filter)										
Denoised estimate $\hat{x}_0$	20.86	28.86	−0.76 [−0.97, −0.55]	0.929	0.059	13.00	21.82	−0.71 [−0.86, −0.56]	0.830	0.167
Noise estimate $\hat{\epsilon}$	20.85	28.59	−1.02 [−1.34, −0.72]	0.926	0.062	13.00	21.89	−0.63 [−0.82, −0.46]	0.826	0.175
Velocity v	20.86	28.86	−0.76 [−0.97, −0.55]	0.929	0.059	13.00	21.93	−0.59 [−0.76, −0.44]	0.831	0.167

Table S6. **Ablations** on DrawBench-200 (held out; all design choices were made on a separate 64-prompt COCO development set), FLUX.1-dev, 1024^2, 50 steps, guidance 3.5. Fidelity to the 50-step reference of the same seed. Every row uses the same SEA gate threshold δ; NFE is the mean number of full transformer evaluations per image, counted by hooks on all 57 blocks (the correction calls no transformer block; it re-evaluates only the embedding and output layers). ΔPSNR: paired difference to KalSea over the 200 prompts, 95% percentile-bootstrap CI (10k resamples). Mean NFE of every row is within 0.06 / 0.00 of KalSea at δ=0.3 / 0.6 (matched compute). A second, independent run of KalSea (the DCT filter's atomic reductions are not bitwise reproducible) differs by δ=0.3: +0.02 dB [−0.04, +0.08]; δ=0.6: +0.04 dB [−0.00, +0.10]. Extended version of the main paper's Tab. 3.

E. Development-set results

All design decisions were made on a development set of 64 COCO captions [6] (seed = prompt index), disjoint from the 16 calibration captions and from DrawBench; DrawBench was only used to score finished configurations. During development we used the first half of the captions (A, prompts 0–31) and confirmed on the second half (B, 32–63). The development set used the thresholds $\delta = 0.37$ (18.0 NFE) and 0.6 (13.0 NFE). Tab. S14 gives the gains of PCGS for both forecasters and for reuse on both halves; Tab. S15 the development-set ablations. On this set, the per-element filter with PCGS was the best configuration at 18 NFE (27.94 vs. 27.58 dB for the DCT filter with PCGS); on DrawBench the two are tied (Tab. S7). The RTS gains and the linear interpolation were tied on the development set for the per-element filter (interpolation minus RTS: −0.13 dB [−0.31, 0.03] at 18 NFE and +0.07 dB [−0.05, 0.20] at 13 NFE, Tab. S15), which is where we chose the RTS gains; on DrawBench the interpolation is better by 0.14 dB at 20.8 NFE and tied at 13 NFE (main paper, Sec. 4.4), and we did not change the method based on the test set.

F. What did not help

Forecasting a sampler quantity instead of the residual. A forecaster can track a quantity of the sampler, the clean-image estimate $\hat{\mathbf{x}}_0$, the noise estimate $\hat{\boldsymbol{\epsilon}}$ or the velocity $\mathbf{v}$, and reconstruct the velocity from the current latent on skipped steps; for $\hat{\mathbf{x}}_0$ and $\hat{\boldsymbol{\epsilon}}$ this feeds the latent's own drift back into the output. Open loop, along uncached trajectories, the $\hat{\boldsymbol{\epsilon}}$ forecast is the most accurate (Tab. S12). In closed-loop sampling, however, no target is better than the residual. On the development set all differences are within 0.13 dB with intervals that include zero (Tab. S15); on DrawBench with

Table S7. **Gap smoothing helps both forecaster variants.** FLUX.1-dev on DrawBench-200 at matched NFE; rows: Kalman forecaster on DCT bands (with band-specific warped time) or per element, without (✗) or with (✓) predictor-corrector gap smoothing (PCGS). Δ: paired PSNR gain of PCGS with 95% bootstrap CI over prompts. With PCGS, the two forecasters do not differ significantly (DCT minus per-element: +0.02 dB [−0.09, +0.15] at 18.1 NFE, +0.01 dB [−0.13, +0.15] at 13 NFE). Best per column in bold.

		18.1 NFE (δ=0.37)			13 NFE (δ=0.6)		
Forecaster	PCGS	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
DCT bands	✗	26.94	0.907	0.080	21.92	0.826	0.172
	✓	**27.72**	**0.916**	**0.070**	**22.53**	0.842	0.154
	Δ	+0.78 dB [+0.60, +0.97]			+0.61 dB [+0.48, +0.75]		
Per element	✗	26.87	0.907	0.080	21.94	0.829	0.169
	✓	27.70	0.915	0.071	22.52	**0.843**	**0.153**
	Δ	+0.83 dB [+0.66, +1.01]			+0.58 dB [+0.44, +0.73]		

Table S8. **PCGS on different predictors** (FLUX.1-dev, DrawBench-200; same SEA gate and threshold, matched NFE). PSNR without and with PCGS, paired gain (mean and 95% bootstrap CI over prompts) and the fraction of prompts on which PCGS raises PSNR. Smoother: the Kalman forecasters use their RTS gains; reuse and linear extrapolation have no state-space model and use the forecast plus a linearly weighted innovation, which for reuse is the linear interpolation from $\mathbf{r}_a$ to $\mathbf{r}_b$ (Sec. B). All other parts of PCGS (re-integration, continuation, gate refresh) are the same.

		20.8 NFE (δ=0.3)				13 NFE (δ=0.6)			
Predictor	Smoother	without	with	gain (dB)	win	without	with	gain (dB)	win
Reuse (SeaCache)	lin. interp.	26.89	28.58	+1.69 [+1.51, +1.88]	92%	21.26	22.00	+0.74 [+0.62, +0.86]	78%
Linear extrapolation	fcst. + lin. innov.	28.51	29.66	+1.15 [+0.92, +1.38]	82%	21.80	22.59	+0.80 [+0.62, +0.98]	78%
Kalman, DCT bands (KalSea w/o PCGS → KalSea)	RTS	28.79	29.62	+0.83 [+0.63, +1.02]	81%	21.91	22.53	+0.61 [+0.48, +0.76]	78%
Kalman, calibration-free per element	RTS	28.75	29.60	+0.86 [+0.67, +1.05]	82%	22.00	22.55	+0.55 [+0.42, +0.68]	74%

Table S9. **All budgets of the main paper's Fig. 3** (FLUX.1-dev, DrawBench-200, metrics w.r.t. the 50-step reference; same code, seeds and hardware for all methods; SeaCache: our re-run of its released gate with residual reuse). The SEA-gate threshold δ sets the budget and is shared by the three methods; NFE is the mean of KalSea (the three methods differ by at most 0.06 NFE at any δ). ΔPSNR: paired per-prompt gain of KalSea with 95% bootstrap CI over prompts. Best per row and metric in bold.

		SeaCache			KalSea w/o PCGS			KalSea			ΔPSNR of KalSea (dB)	
δ	NFE	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	vs. SeaCache	vs. KalSea w/o PCGS
0.2	27.00	30.52	0.944	0.046	32.89	0.960	0.030	**34.41**	**0.968**	**0.025**	+3.89 [+3.49, +4.28]	+1.52 [+1.30, +1.75]
0.3	20.84	26.89	0.904	0.086	28.79	0.927	0.060	**29.62**	**0.935**	**0.053**	+2.73 [+2.34, +3.12]	+0.83 [+0.63, +1.02]
0.37	18.14	25.33	0.880	0.113	26.91	0.907	0.080	**27.72**	**0.916**	**0.070**	+2.39 [+2.03, +2.75]	+0.81 [+0.63, +1.00]
0.5	15.00	22.79	0.835	0.168	23.79	0.862	0.127	**24.58**	**0.877**	**0.112**	+1.78 [+1.48, +2.09]	+0.79 [+0.61, +0.97]
0.6	13.00	21.26	0.800	0.214	21.91	0.826	0.172	**22.53**	**0.842**	**0.154**	+1.27 [+1.02, +1.52]	+0.61 [+0.48, +0.76]
0.8	11.00	20.19	0.768	0.261	20.30	0.779	0.225	**20.93**	**0.800**	**0.199**	+0.74 [+0.46, +1.04]	+0.64 [+0.51, +0.77]
1.0	9.00	19.46	0.738	0.305	19.36	0.738	0.272	**19.67**	**0.747**	**0.263**	+0.21 [−0.005, +0.42]	+0.31 [+0.24, +0.39]

DCT filters and without correction, the residual minus the target is −0.07 dB [−0.23, 0.08] and +0.09 dB [−0.01, 0.20] for $\hat{\mathbf{x}}_0$, +0.20 dB [−0.08, 0.48] and +0.02 dB [−0.13, 0.16] for $\hat{\boldsymbol{\epsilon}}$, and −0.07 dB [−0.22, 0.08] and −0.02 dB [−0.12, 0.09] for $\mathbf{v}$ (at 20.8 and 13 NFE; absolute values in Tab. S6, group (f)). A mixture of $\hat{\mathbf{x}}_0$ and $\hat{\boldsymbol{\epsilon}}$ with Wiener weights was not better either.

Using the innovation to decide when to compute. Instead of correcting the gap, the forecast error can set the length of the next gap: an error controller measured the relative output error of the forecast at each computed step (one extra output-layer evaluation) and scaled the next gap by $\mathrm{clip}_{[0.5,2]}\big((\epsilon_{\mathrm{tol}}/\mathrm{err})^{1/2}\big)$, replacing the SEA gate. On the development set it raised PSNR over the SEA-gated per-element KalSea w/o PCGS at a similar mean NFE (+2.76 dB [1.75, 3.80] at 13.36 vs. 13 NFE), but the gain came from a different allocation of the computed steps rather than from adaptivity: a fixed schedule with the controller's typical allocation, (0, 1, 2, 3, 4, 6, 8, 11, 16, 23, 31, 42, 49), was as good as the controller (−0.26 dB [−0.93, 0.44] for the controller relative to it, 30 prompts), and a predictive variant driven by the filter covariance alone was much worse (−5.07 dB [−6.01, −4.17] at 13 NFE relative to KalSea w/o PCGS).

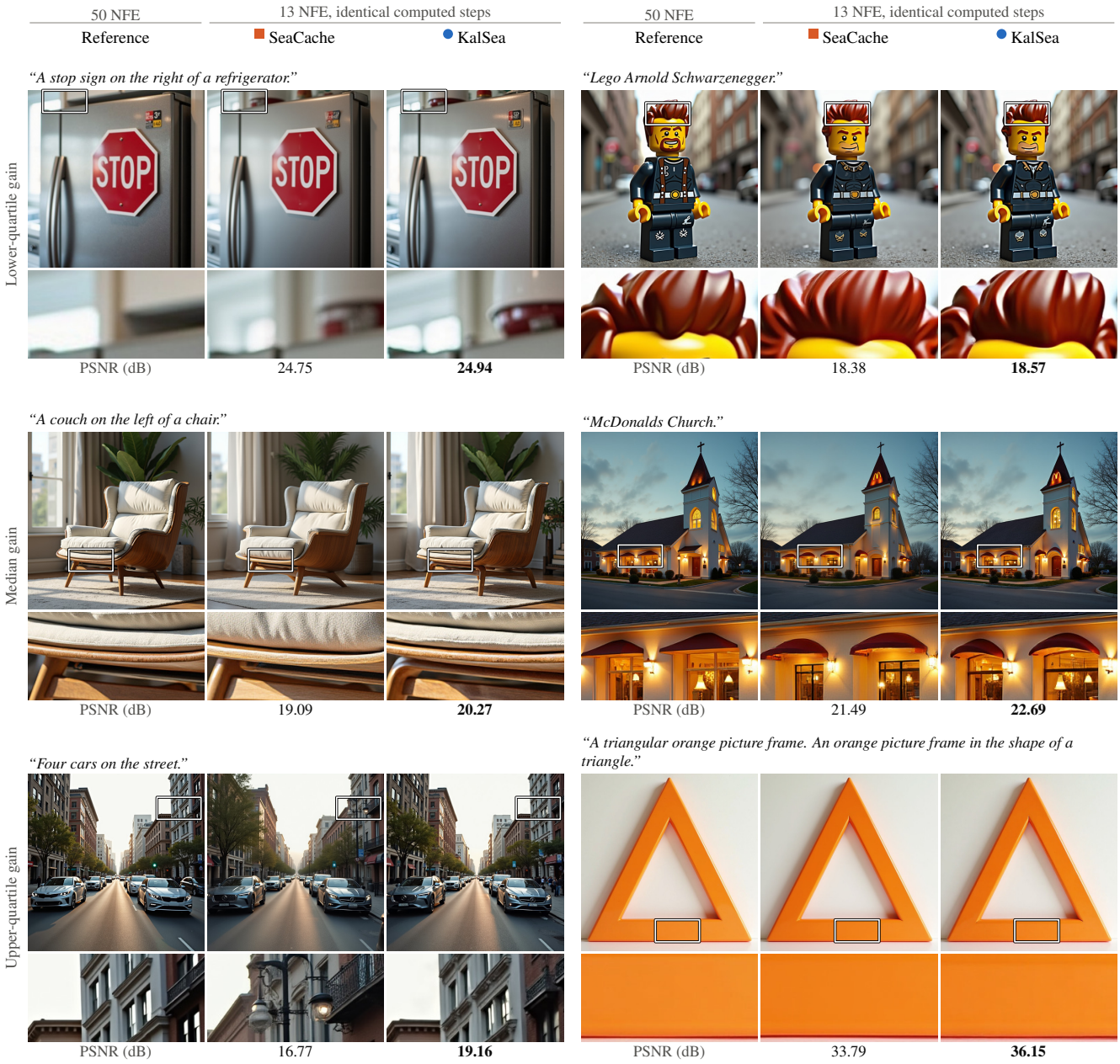


Figure S1. **Qualitative comparison at 13 NFE** (FLUX.1-dev, DrawBench, $\delta = 0.6$, $3.8\times$ fewer transformer evaluations than the 50-step reference). The prompts are selected by a fixed rule, not by hand: among the 188 of 200 prompts on which SeaCache and KalSea computed the identical set of steps, we compute the per-prompt PSNR gain of KalSea over SeaCache and show the two prompts whose gain is closest to the lower quartile (+0.19 dB; top row), the two closest to the median (+1.19 dB; middle row) and the two closest to the upper quartile (+2.40 dB; bottom row). Below each image, a fixed 256×128-pixel crop (of 1024×1024) is magnified; it is placed where the two accelerated samples differ most (largest mean absolute difference), without looking at the reference. Numbers: PSNR (dB) with respect to the reference, best in bold.

The per-image NFE of the controller also varied (standard deviation 1.6–2.4), so its budget was matched only on average. The fixed schedule was derived on the same development prompts and was not evaluated on DrawBench. Which steps to compute is thus a separate question; all methods in this paper share SeaCache's gate.

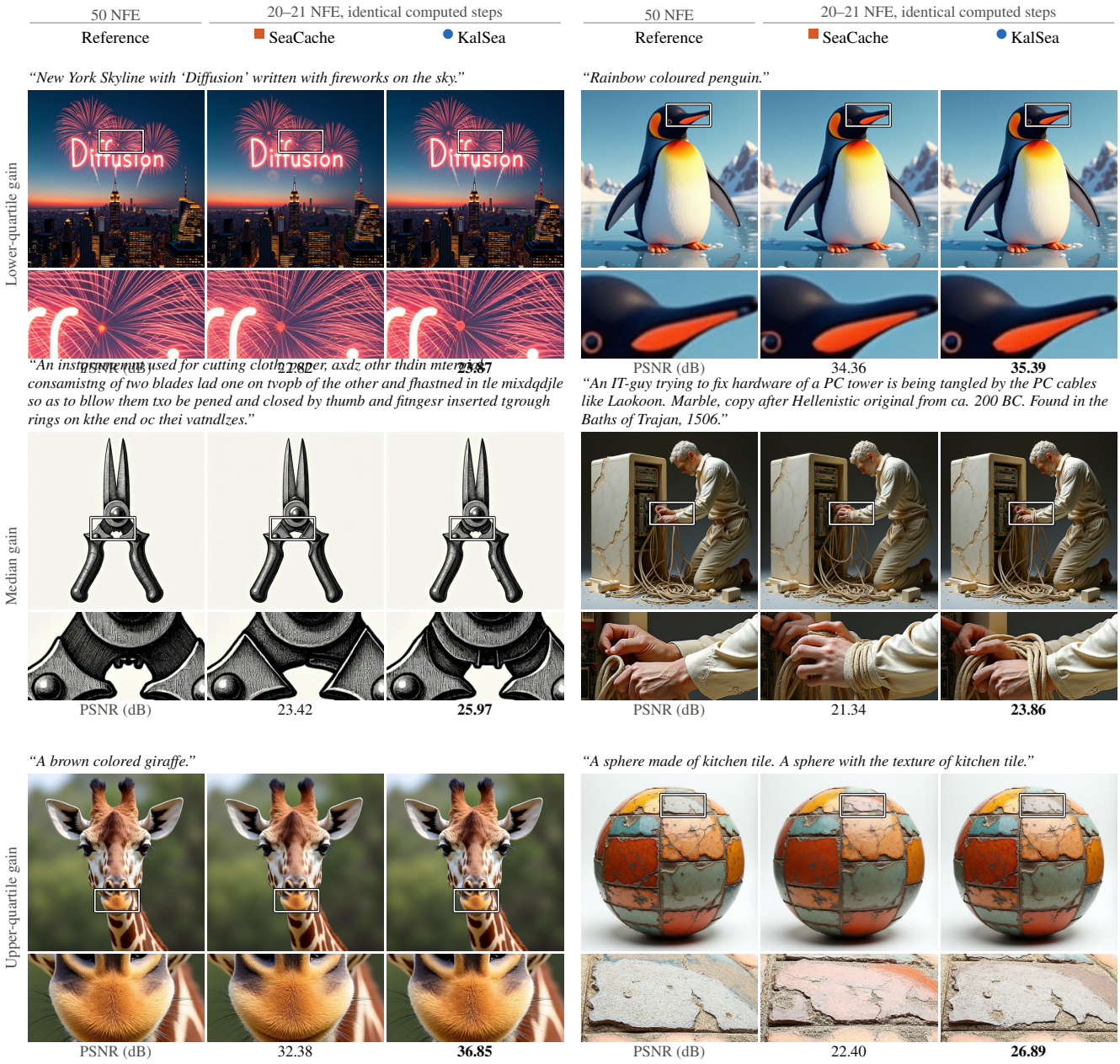


Figure S2. **Qualitative comparison at 20–21 NFE (mean 20.8)** (FLUX.1-dev, DrawBench, $\delta = 0.3$, 2.4× fewer transformer evaluations than the 50-step reference). The prompts are selected by a fixed rule, not by hand: among the 192 of 200 prompts on which SeaCache and KalSea computed the identical set of steps, we compute the per-prompt PSNR gain of KalSea over SeaCache and show the two prompts whose gain is closest to the lower quartile (+1.04 dB; top row), the two closest to the median (+2.53 dB; middle row) and the two closest to the upper quartile (+4.48 dB; bottom row). Below each image, a fixed 256 × 128-pixel crop (of 1024 × 1024) is magnified; it is placed where the two accelerated samples differ most (largest mean absolute difference), without looking at the reference. Numbers: PSNR (dB) with respect to the reference, best in bold.

G. Video generation

G.1. Protocol and implementation

Protocol. We follow the video protocol of SeaCache [2]: the 944 unique prompts of VBench's full prompt list [4] (946

Table S10. **Forecasting without correction across budgets** (FLUX.1-dev, DrawBench-200, the thresholds of the main paper's Fig. 3). PSNR of linear extrapolation of the residual and paired PSNR difference to reuse (SeaCache, same gate and threshold) of linear extrapolation and of the Kalman forecast (KalSea w/o PCGS), mean and 95% CI over prompts. The advantage of forecasting over reuse shrinks as the gaps grow; PCGS improves on the Kalman forecast at every budget (Tab. S9).

		Lin. extr.	ΔPSNR vs. reuse (dB)	
δ	NFE	PSNR	lin. extrapolation	Kalman (KalSea w/o PCGS)
0.2	27.00	32.27	+1.75 [+1.43, +2.06]	+2.37 [+2.02, +2.71]
0.3	20.84	28.51	+1.61 [+1.28, +1.94]	+1.90 [+1.54, +2.26]
0.37	18.14	26.66	+1.33 [+1.05, +1.63]	+1.58 [+1.27, +1.90]
0.5	15.00	23.58	+0.79 [+0.50, +1.05]	+0.99 [+0.71, +1.28]
0.6	13.00	21.80	+0.54 [+0.28, +0.80]	+0.66 [+0.42, +0.88]
0.8	11.00	19.61	−0.58 [−0.93, −0.24]	+0.11 [−0.14, +0.34]
1.0	9.00	18.70	−0.77 [−1.05, −0.50]	−0.10 [−0.31, +0.10]

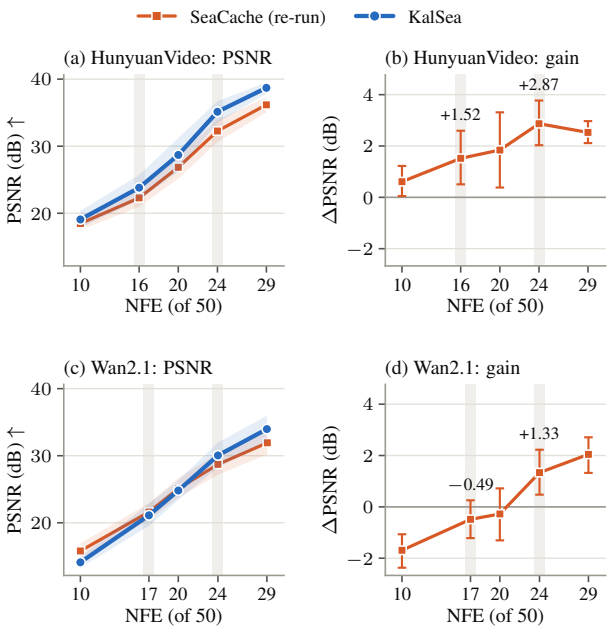


Figure S3. **Budget sweep on video** (20 VBench prompts per model, those of Tab. S19; HunyuanVideo with the corrected gate). **(a, c)** PSNR w.r.t. the 50-step reference vs. NFE for our same-setup rerun of SeaCache and for KalSea, which share the SEA gate and its threshold (HunyuanVideo $\delta \in \{0.195, 0.2575, 0.35, 0.48, 0.865\}$, Wan2.1 $\delta \in \{0.14, 0.2, 0.265, 0.35, 0.815\}$); lines: means, shaded: 95% bootstrap CIs. **(b, d)** Paired PSNR gain of KalSea over SeaCache (mean, 95% CI); gray bands: the budgets of the main paper's Tab. 2. On HunyuanVideo, KalSea is better at every budget (+0.61 dB [+0.05, +1.22] at 10 NFE, +2.53 dB [+2.11, +2.97] at 29). On Wan2.1 the gain shrinks with the budget: it is significant at 29 and 24 NFE, not significant at 20 and 17 NFE, and negative at 10 NFE (−1.69 dB [−2.37, −1.07]). Both methods computed the same steps on at least 19 (HunyuanVideo) and 15 (Wan2.1) of the 20 prompts at every threshold.

lines with two duplicates), one video per prompt, seed 42 for every prompt and method, 50 sampling steps, and fidelity to the uncached 50-step video of the same prompt and seed.

- HunyuanVideo [5] (official code): 480×640, 65 frames, Euler flow sampler with shift 7 and reversed flow, embedded guidance 6, a single (guidance-distilled) pass per step, bf16; videos saved at 24 fps.
- Wan2.1-T2V-1.3B [13] (official code): 832×480, 65 frames, UniPC [15] with shift 5, classifier-free guidance [3] with scale 5 and the default negative prompt, bf16; 16 fps. SeaCache's text states 65 frames for both models, but its Wan2.1 compute corresponds to 81 frames: its 8214 TFLOPs for 50 steps (164.3 per step) match the linear-layer FLOPs of this model at 832×480 and 81 frames (164.2 per step, both guidance passes), whereas 65 frames give 133.0. Its published Wan2.1 results may thus come from 81-frame videos; we use the 65 frames of its text.
- Gate: the official SEA gate of each model ($\beta = 3$, filter over frames, height and width), first and last step always computed, thresholds passed as parameters. With the gate as released, our SeaCache path reproduces the outputs of SeaCache's released scripts bit for bit on our test prompts (both models). On HunyuanVideo, every gated method reads the sampler's noise level (Sec. G.2).
- NFE: full transformer evaluations per video, counted by forward hooks on every block and checked against the gate's record (no mismatch in any video). On Wan2.1, the conditional and unconditional passes keep separate gate accumulators, but their gate distances were identical in every video, so both passes computed the same steps; a computed step runs both passes and counts once.
- Metrics: PSNR, SSIM and LPIPS on the decoded MP4 frames, as in TeaCache's video metric code [7]: PSNR on 8-bit RGB per frame (capped at 100 dB), SSIM with an 11×11 Gaussian window ($\sigma = 1.5$) averaged over the RGB channels, and LPIPS with AlexNet features on inputs in $[-1, 1]$; frames are averaged per video, then videos. Paired 95% bootstrap intervals over videos (10,000 resamples, seed 0).

Ports of the forecaster and of PCGS. Both models use the calibration-free per-element filter (Sec. A.3), one filter per guidance branch (one for HunyuanVideo, two for Wan2.1), updated under deterministic algorithms so that runs are reproducible. HunyuanVideo samples with Euler steps, so PCGS applies as for FLUX.1-dev, with the patch embedding and the final layer as Emb and Out. For Wan2.1, whose UniPC solver is multistep, the solver state is saved after each computed step a; when step b closes the gap, both branches are smoothed, the state is restored and the solver steps $a+1, \ldots, b-1$ are replayed with the corrected guided

Table S11. **Size of the latent correction** of KalSea on FLUX.1-dev (DrawBench-200): relative correction $\|\mathbf{x}'_b - \mathbf{x}_b\|/\|\mathbf{x}_b\|$ at the computed step b that closes a gap, averaged over all gaps, over the gaps of length one, and over the gaps of the largest length; output-layer evaluations (Emb and Out, counted) and gate refreshes per image. Bottom: per gap of the most common schedule at δ=0.6 (steps 0-based).

		correction (%)			per image	
δ	NFE	all gaps	length 1	longest	Emb, Out	refresh
0.2	27.00	0.38	0.28	0.46 (2)	38.1	15.0
0.3	20.84	0.70	0.17	1.72 (4)	42.2	12.8
0.37	18.14	1.00	0.21	2.31 (5)	44.0	11.2
0.5	15.00	1.56	0.33	2.78 (6)	45.3	10.0
0.6	13.00	2.00	0.71	4.12 (7)	47.0	9.0
0.8	11.00	2.93	0.59	9.14 (10)	47.7	8.0
1.0	9.00	4.37	0.29	12.43 (12)	48.0	6.0

δ=0.6, *schedule (0, 1, 2, 4, 6, 9, 13, 18, 25, 33, 41, 47, 49), 147 prompts*

gap	2–4	4–6	6–9	9–13	13–18	18–25	25–33	33–41	41–47	47–49
corr. (%)	0.24	0.22	0.42	0.64	1.07	2.12	3.36	5.20	6.08	1.83

Table S12. **Open-loop forecast error inside long gaps** (development set, six uncached 50-step trajectories). Each forecaster observes the trajectory only at the steps of the most common 13-NFE schedule of the SEA gate, (0, 1, 2, 4, 6, 9, 13, 18, 25, 33, 41, 47, 49); at the skipped steps of its two seven-step gaps we report the relative error $\|\hat{\mathbf{v}}_i - \mathbf{v}_i\|/\|\mathbf{v}_i\|$ (%) of the velocity obtained from the forecast, along the uncached trajectory (no error feedback). Targets: block-stack residual $\mathbf{r}$ (per-element or DCT filter, as in KalSea) and, with per-element filters, the clean-image estimate $\hat{\mathbf{x}}_0$, the noise estimate $\hat{\boldsymbol{\epsilon}}$ and the velocity $\mathbf{v}$ (sampler quantities, see Sec. F).

	gap 25–33 (steps 26–32)					gap 33–41 (steps 34–40)				
k	$\mathbf{r}$ (elem.)	$\mathbf{r}$ (DCT)	$\hat{\mathbf{x}}_0$	$\hat{\boldsymbol{\epsilon}}$	$\mathbf{v}$	$\mathbf{r}$ (elem.)	$\mathbf{r}$ (DCT)	$\hat{\mathbf{x}}_0$	$\hat{\boldsymbol{\epsilon}}$	$\mathbf{v}$
1	4.9	5.1	5.0	4.4	4.8	4.2	4.5	4.3	3.9	4.1
2	8.5	9.2	8.9	7.2	8.4	7.3	8.2	7.7	6.5	7.2
3	11.8	13.0	12.6	9.5	11.6	10.3	11.8	11.2	8.8	10.1
4	14.8	16.6	16.3	11.4	14.6	13.3	15.4	15.0	10.9	12.9
5	17.7	20.2	20.2	13.2	17.7	15.9	18.7	18.7	12.7	15.6
6	20.3	23.5	23.8	14.5	20.3	18.7	22.1	23.0	14.6	18.3
7	22.8	26.8	27.7	15.9	23.1	21.6	25.6	27.8	16.7	21.2

Table S13. **Prompts of the main paper's Fig. 1** (FLUX.1-dev, 1024×1024, 50 steps, guidance 3.5, KalSea at $\delta = 0.2$, 27 of 50 transformer evaluations per image). Four seeds were generated per prompt and one was picked by eye for legible lettering.

Tile	Seed	Prompt
C	2030	A giant capital letter "C" made of lush green ivy and small white flowers, standing alone in a sunlit garden, centered, sharp photograph
V	2032	A giant capital letter "V" carved from clear blue ice, standing on a snowy mountain ridge under a blue sky, centered, sharp photograph
P	2038	A giant capital letter "P" made of glowing pink neon tubes mounted on a dark brick wall at night, centered, sharp photograph
R	2040	A giant capital letter "R" sculpted from golden sand on a beach at sunset, centered, sharp photograph
20	2043	The number "20" in large white painted digits on the hull of a red wooden fishing boat in a harbor, centered, sharp photograph
KalSea	2049	A large roadside billboard that reads "KalSea" in bold dark blue letters on a white background, with the ocean and a clear blue sky behind it, the wide billboard is centered in the frame, sharp photograph
27	2051	A wooden lifeguard tower on a sandy beach with the number "27" painted in large white digits on its side, centered, sharp photograph

outputs $\mathbf{u}'_i + w(\mathbf{c}'_i - \mathbf{u}'_i)$, where $\mathbf{c}'_i$ and $\mathbf{u}'_i$ are obtained from the patch embedding of the replayed latent, the smoothed residual of the branch and the output head, and w is the guidance scale. Sampling then continues from the replayed latent and solver state, with the step-b outputs recomputed from the corrected latent and the fresh residuals. In our tests on both models, the identity smoother reproduces the uncorrected forecaster bitwise, and for Wan2.1 a corrected replay equals a fresh UniPC run with the corrected outputs bitwise.

G.2. HunyuanVideo gate

The released SeaCache script for HunyuanVideo binds the scheduler that exists when the model is loaded (noise levels from 1 to 0 in steps of $1/1000$) to the gate, while sampling

Table S14. **Development set** (64 COCO captions, disjoint from DrawBench and from the 16 calibration captions; seed = prompt index). Paired PSNR gain (dB, 95% bootstrap CI) on all 64 prompts and on the halves used for development (A, prompts 0–31) and confirmation (B, 32–63); PSNR: first method, all 64 prompts. Rows: (1) PCGS on the DCT filter (KalSea) vs. KalSea w/o PCGS; (2) PCGS on the per-element filter vs. the same filter; (3) reuse with PCGS (linear smoother) vs. reuse (SeaCache); (4) second run of KalSea w/o PCGS vs. first run (noise floor). These numbers guided the design; they are not test results.

	PSNR	all 64	half A	half B
δ=0.37 *(18.0 NFE)*				
(1)	27.58	+0.75 [+0.50, +1.00]	+0.42 [+0.11, +0.76]	+1.07 [+0.74, +1.44]
(2)	27.94	+1.09 [+0.83, +1.38]	+0.95 [+0.56, +1.39]	+1.23 [+0.89, +1.62]
(3)	26.62	+1.31 [+1.00, +1.59]	+1.05 [+0.54, +1.49]	+1.56 [+1.22, +1.91]
(4)	26.79	−0.05 [−0.11, +0.002]	−0.09 [−0.19, +0.01]	−0.02 [−0.07, +0.03]
δ=0.6 *(13.0 NFE)*				
(1)	22.21	+0.73 [+0.46, +1.02]	+0.59 [+0.18, +1.09]	+0.87 [+0.59, +1.18]
(2)	22.33	+0.63 [+0.34, +0.92]	+0.45 [+0.06, +0.94]	+0.80 [+0.46, +1.16]
(3)	21.65	+0.77 [+0.57, +0.97]	+0.75 [+0.48, +1.06]	+0.78 [+0.51, +1.05]
(4)	21.51	+0.03 [−0.01, +0.09]	+0.08 [+0.003, +0.18]	−0.01 [−0.03, −0.001]

Table S15. **Development-set ablations** (COCO-64; per-element filter unless noted; no correction for the forecasting targets). Paired PSNR difference (dB, 95% CI) over the n prompts common to both runs (some runs of this round stopped before 64 prompts).

Comparison	18.0 NFE		13.0 NFE	
	ΔPSNR	n	ΔPSNR	n
Linear interpolation $\mathbf{r}_a \to \mathbf{r}_b$ vs. RTS gains	−0.13 [−0.31, +0.03]	63	+0.07 [−0.05, +0.20]	62
Quadratic interpolation vs. RTS gains	−0.23 [−0.39, −0.09]	63	−0.05 [−0.12, +0.01]	62
Batched vs. exact re-integration	−0.36 [−0.61, −0.18]	63	−0.10 [−0.26, +0.02]	62
No gate refresh vs. refresh	+0.02 [−0.002, +0.04]	63	−0.003 [−0.01, +0.00]	62
Calibration-free vs. calibrated filter, no correction	−0.03 [−0.22, +0.15]	62	+0.07 [+0.02, +0.12]	62
PCGS on the calibration-free filter vs. the same filter	+0.83 [+0.61, +1.05]	62	+0.51 [+0.19, +0.82]	62
Target $\hat{\mathbf{x}}_0$ vs. residual (per element)	−0.06 [−0.32, +0.20]	60	−0.10 [−0.24, +0.04]	58
Target $\hat{\mathbf{x}}_0$ vs. residual (DCT)	+0.12 [−0.13, +0.40]	59	−0.04 [−0.14, +0.07]	57
Target $\hat{\boldsymbol{\epsilon}}$ vs. residual (per element)	−0.07 [−0.32, +0.18]	60	−0.12 [−0.44, +0.19]	57
Target $\mathbf{v}$ vs. residual (per element)	−0.10 [−0.30, +0.08]	60	−0.09 [−0.21, +0.03]	57
Target $\mathbf{v}$ vs. residual (DCT)	−0.13 [−0.29, +0.03]	59	+0.01 [−0.08, +0.10]	57
Target: Wiener mix of $\hat{\mathbf{x}}_0$, $\hat{\boldsymbol{\epsilon}}$ vs. residual (per element)	−0.07 [−0.27, +0.12]	60	−0.06 [−0.20, +0.07]	57
Target: Wiener mix of $\hat{\mathbf{x}}_0$, $\hat{\boldsymbol{\epsilon}}$ vs. residual (DCT)	−0.03 [−0.22, +0.20]	59	+0.03 [−0.08, +0.12]	57

creates a new scheduler with the shifted 50-step schedule. Its SEA filter therefore uses $\sigma = 1 - i/1000$ at step i, a noise level near one at every step, instead of the sampling schedule. This matters most at step 1, whose gate distance is 0.185–0.221 over the 944 prompts with the stale noise level and 0.661–0.668 with the sampler's. With the released gate, step 1 is thus skipped on most prompts once $\delta \gtrsim 0.21$: on 4 prompts at SeaCache's published $\delta = 0.19$, on 780 at $\delta = 0.21$ and on all of them above 0.222. Skipping it is costly: at $\delta = 0.207$ (23.8 NFE), the 559 prompts that skip step 1 score 12.4 dB (SeaCache) and 14.0 dB (KalSea) below the others, and the corrected gate raises their PSNR by 11.4 and 13.1 dB at the same budget (others: −0.5 and −0.4 dB). We therefore let the gate of every gated method (SeaCache, KalSea and all ablations) read the noise levels of the sampling schedule; at step 0 of each of the 3776 videos of the main comparison, we checked that the gate's noise levels equal the sampler's. The re-integration of PCGS always uses the sampling schedule. Tab. S16 compares the two gates on all prompts at matched NFE: the correction raises the PSNR of both methods by several dB, and KalSea improves on SeaCache with either gate, more with the corrected one.

G.3. Budgets and thresholds

Tab. S17 lists the threshold or stride of every row of the main paper's Tab. 2. We match NFE, not thresholds: SeaCache reports TFLOPs, which we convert to NFE, and the threshold that reaches a given NFE depends on the setup.

- Wan2.1: SeaCache's thresholds $\delta = 0.2$ and 0.35 compute 24.0 and 17.0 steps per video in our setup, the NFE implied by its TFLOPs.
- HunyuanVideo, released gate (Tab. S16 only): SeaCache's $\delta = 0.19$ computes 25.8 steps on our 20 pilot prompts, more than the 24.0 implied by its TFLOPs, so we use $\delta = 0.207$ (23.8 NFE) and its $\delta = 0.35$ (16.0 NFE; published: 16.4).
- HunyuanVideo, corrected gate: the gate distances change, so we set new thresholds by replaying the gate on the distances recorded in a corrected-gate run of SeaCache

Table S16. **HunyuanVideo gate: as released vs. corrected** (944 VBench prompts). Released: SeaCache's gate with the load-time scheduler (noise level $1 - i/1000$ at step i); corrected: the same gate with the noise levels of the sampling schedule. Thresholds are set per gate for the same NFE. Both methods use the same gate in each column. Δ: paired difference, mean and 95% bootstrap CI over videos (win: fraction of videos where the first term is higher).

	≈24 NFE		16 NFE	
Gate	released	corrected	released	corrected
Threshold δ	0.207	0.2575	0.35	0.48
NFE (SeaCache/ KalSea)	23.81 / 23.81	24.03 / 24.03	16.00 / 16.00	16.00 / 16.00
Videos computing step 1	385/944	944/944	0/944	944/944
SeaCache: PSNR / SSIM / LPIPS	25.91 / 0.819 / 0.156	32.43 / 0.929 / 0.048	20.52 / 0.727 / 0.246	24.53 / 0.813 / 0.155
KalSea: PSNR / SSIM / LPIPS	28.03 / 0.844 / 0.128	35.60 / 0.951 / 0.028	21.88 / 0.761 / 0.203	26.45 / 0.846 / 0.117
ΔPSNR KalSea −SeaCache	+2.12 [+1.94, +2.30]	+3.17 [+2.99, +3.35]	+1.36 [+1.20, +1.53]	+1.92 [+1.70, +2.14]
win	79%	90%	67%	72%
ΔPSNR corrected−released, SeaCache	+6.52 [+6.06, +6.97], win 74%		+4.01 [+3.76, +4.28], win 87%	
ΔPSNR corrected−released, KalSea	+7.57 [+7.05, +8.08], win 78%		+4.57 [+4.22, +4.92], win 80%	

on the 20 pilot prompts: $\delta = 0.2575$, the middle of the range that gives 24.0 NFE, and $\delta = 0.4637$ for 16.4. On all prompts, $\delta = 0.2575$ gives 24.0 NFE, but $\delta = 0.4637$ gives 16.7, more than SeaCache's 16.4. We therefore ran both methods again at $\delta = 0.48$, which computes exactly 16 steps for every prompt, and report these runs. At $\delta = 0.4637$ (Tab. S18, bottom), with more compute than published, our re-run of SeaCache reaches 26.00 dB, still below the published 26.46 dB, and KalSea improves on it by 2.76 dB [2.55, 2.97].

G.4. Same-setup comparisons

Tab. S18 compares KalSea with our re-run of SeaCache on all prompts, with the same code, seeds, GPUs and gate. On HunyuanVideo, KalSea improves PSNR by 3.17 dB [2.99, 3.35] and 1.92 dB [1.70, 2.14] at 24 and 16 NFE, on 90% and 72% of the videos, with better SSIM and LPIPS. On Wan2.1, it improves PSNR by 2.27 dB [2.13, 2.42] at 24 NFE (83% of the videos) but only by 0.33 dB [0.16, 0.49] at 17 NFE, where it has the higher PSNR on 50% of the videos; SSIM and LPIPS improve on 70% and 76% of them. The two methods computed the same steps on 936 (24 NFE) and 936 (16 NFE) of the 944 HunyuanVideo prompts and on 896 (24 NFE) and 729 (17 NFE) of the Wan2.1 prompts, and the per-video NFE never differs by more than one.

G.5. Ablations and budget sweep

Ablations. On 20 prompts per model, drawn at random (stratified by VBench dimension) for our pilot runs before the full runs, we repeat the ablations of the main paper's Tab. 3 that concern the corrector and the predictor (Tab. S19). All variants share the gate, threshold and NFE of KalSea. These prompts are not held out: the pilot runs used them, and on HunyuanVideo they also set the corrected-gate thresholds. On HunyuanVideo the results agree with FLUX.1-dev; on Wan2.1 they are weaker:

- The shape of the gains hardly matters: linear interpolation from $\mathbf{r}_a$ to $\mathbf{r}_b$ instead of the RTS gains changes PSNR by +0.05 dB [−0.65, 0.61] and +0.20 dB [−0.40, 0.75] on HunyuanVideo and by −0.09 dB [−0.31, 0.17] and +0.04 dB [−0.06, 0.13] on Wan2.1.
- The gain comes from re-integrating the gap: feeding the smoothed residuals only into the filter state changes PSNR by −1.28 dB [−1.83, −0.79] and −1.10 dB [−1.60, −0.64] on HunyuanVideo; on Wan2.1 the difference is small (−0.34 dB [−0.77, 0.07] and −0.04 dB [−0.22, 0.15]).
- PCGS is not tied to the Kalman forecaster: it improves reuse on both models at both budgets (HunyuanVideo: 2.33 dB [1.88, 2.77] and 1.15 dB [0.57, 1.83]; Wan2.1: 1.45 dB [1.13, 1.77] and 0.55 dB [0.27, 0.86]), and linear extrapolation on HunyuanVideo (1.49 dB [0.92, 2.15] and 1.10 dB [0.58, 1.67]) but not significantly on Wan2.1 (0.08 dB [−0.46, 0.62] and 0.17 dB [−0.12, 0.44]).
- On Wan2.1 at 17 NFE, the predictor is the weak part: on these prompts, KalSea differs from SeaCache by −0.49 dB [−1.22, 0.26] (on all 944 prompts: +0.33 dB [0.16, 0.49]), whereas reuse with PCGS exceeds KalSea by 1.04 dB [0.38, 1.72].

Budget sweep. Fig. S3 extends the comparison with SeaCache to five budgets per model on the same 20 prompts. The three thresholds per model added to those of Tab. 2 target 10, 20 and 30 NFE and were fixed before these runs: on a grid of step 0.005, among the thresholds whose replayed mean NFE is within one of the target, we chose the one at which the replayed schedules of the two methods agree on the most prompts for every threshold within ±1%. On HunyuanVideo, KalSea is better at every budget, by +0.61 dB [0.05, 1.22] at 10 NFE and +2.53 dB [2.11, 2.97] at 29. On Wan2.1, its gain shrinks with the budget, and at 10 NFE KalSea is −1.69 dB [−2.37, −1.07] relative to SeaCache.

Table S17. **Settings of every row of the main paper's Tab. 2**: threshold or stride and, in parentheses, NFE. †Settings and results published by SeaCache [2] (its Tables 3 and 4); NFE $= 50 \times$ TFLOPs/TFLOPs of its 50-step run. TaylorSeer computes the first three steps and then every S-th step (expansion order 2, as stated by SeaCache). TeaCache and the SEA gate compute a step once the accumulated relative change of the first block's modulated input (rescaled for TeaCache, SEA-filtered for SeaCache and KalSea) exceeds δ; the first and last steps are always computed. §Our re-run of SeaCache's released code with the corrected HunyuanVideo gate (Sec. G.2), which needs its own thresholds for the same NFE; for Wan2.1 the main paper reports the published SeaCache row. Our rows: mean NFE over the 944 prompts.

		HunyuanVideo		Wan2.1 1.3B	
Method	Parameter	$\approx 50\%$	$\approx 30\%$	$\approx 50\%$	$\approx 30\%$
Fewer steps†	sampling steps	25 steps (25.0)	15 steps (15.0)	–	–
TeaCache [7]†	threshold δ (rel. L1)	δ=0.12 (24.9)	δ=0.2 (17.1)	δ=0.09 (25.0)	δ=0.15 (18.0)
TaylorSeer [8]†	stride S	S=2 (26.0)	S=3 (18.0)	S=2 (25.5)	S=3 (18.0)
SeaCache, published†	SEA threshold δ	δ=0.19 (24.0)	δ=0.35 (16.4)	δ=0.2 (24.0)	δ=0.35 (17.0)
SeaCache, our re-run§	SEA threshold δ	δ=0.2575 (24.03)	δ=0.48 (16.00)	–	–
KalSea	SEA threshold δ	δ=0.2575 (24.03)	δ=0.48 (16.00)	δ=0.2 (24.01)	δ=0.35 (17.00)

Table S18. **Same-setup video comparisons** (944 VBench prompts, 65 frames; same code, seeds, GPUs and SEA gate for both methods; HunyuanVideo gate with the sampler's noise level). Δ: KalSea minus SeaCache, mean and 95% paired bootstrap CI over videos; win: fraction of videos with higher PSNR for KalSea; same steps: videos on which both computed exactly the same steps (the gate is closed loop, so the filler can change its decisions; the per-video NFE differs by at most one). Bottom: the first low-budget threshold of HunyuanVideo, which used more than SeaCache's published 16.4 NFE and is therefore not in Tab. 2 (Sec. G.3).

Method	NFE	PSNR↑	SSIM↑	LPIPS↓	ΔPSNR (dB)	ΔSSIM	ΔLPIPS	win	same steps
HunyuanVideo, δ=0.2575 (24.0 NFE)									
SeaCache (re-run)	24.03	32.43	0.929	0.048	+3.17 [+2.99, +3.35]	+0.022 [+0.020, +0.024]	−0.020 [−0.022, −0.019]	89.7%	936/944
KalSea	24.03	35.60	0.951	0.028		(reference of Δ)			
HunyuanVideo, δ=0.48 (16.0 NFE)									
SeaCache (re-run)	16.00	24.53	0.813	0.155	+1.92 [+1.70, +2.14]	+0.033 [+0.029, +0.038]	−0.039 [−0.043, −0.034]	71.9%	936/944
KalSea	16.00	26.45	0.846	0.117		(reference of Δ)			
Wan2.1, δ=0.2 (24.0 NFE)									
SeaCache (re-run)	24.01	29.40	0.931	0.043	+2.27 [+2.13, +2.42]	+0.022 [+0.021, +0.023]	−0.023 [−0.024, −0.022]	82.9%	896/944
KalSea	24.01	31.67	0.953	0.020		(reference of Δ)			
Wan2.1, δ=0.35 (17.0 NFE)									
SeaCache (re-run)	17.00	23.52	0.828	0.118	+0.33 [+0.16, +0.49]	+0.023 [+0.020, +0.027]	−0.029 [−0.032, −0.026]	49.7%	729/944
KalSea	17.00	23.85	0.851	0.090		(reference of Δ)			
HunyuanVideo, δ=0.4637 (16.7 NFE, not in Tab. 2)									
SeaCache (re-run)	16.70	26.00	0.839	0.129	+2.76 [+2.55, +2.97]	+0.043 [+0.040, +0.047]	−0.044 [−0.047, −0.040]	78.4%	915/944
KalSea	16.69	28.76	0.882	0.085		(reference of Δ)			

Together with the ablation at 17 NFE, where reuse with PCGS is better than KalSea, this suggests that on Wan2.1 at small budgets the Kalman forecast of the two guidance branches, not the correction, limits KalSea.

G.6. Video examples

Figs. S4 and S5 show two videos per model at the lower budget of Tab. 2 (HunyuanVideo: $\delta = 0.48$, 16 NFE; Wan2.1: $\delta = 0.35$, 17 NFE), each as five frames spread over the 65-frame clip. The videos are selected by a fixed rule, not by hand: among the prompts on which SeaCache and KalSea computed the identical steps, we compute the per-video PSNR gain of KalSea over SeaCache with respect to the 50-step reference and show the video whose gain is closest to the median and the one closest to the upper quartile of the gains. The latter is also the video example of the main paper's Fig. 4, which shows its middle frame; the FLUX.1-dev example there is selected by the same rule among the DrawBench prompts at $\delta = 0.6$ (13 NFE). On Wan2.1 the median gain is small, in line with the small mean gain at this budget. Note that the per-video PSNR is a mean over all 65 frames, so a single frame can look closer to the reference than the number suggests.

G.7. Reproducibility

The video runs are deterministic. For 7 configurations of both models, the full runs regenerated the 20 prompts of an earlier pilot run and reproduced its videos bit for bit (140 videos: MP4 files, decoded frames and computed steps), in part after code changes in between. After the ablation options were added to the code, the configurations of the main comparison (SeaCache and KalSea at both budgets of both models, one prompt each, 8 videos) again reproduced the videos of the full runs bit for bit. Differences between video

Table S19. **Video ablations** (20 VBench prompts per model, the prompts of our pilot runs; same SEA gate and threshold for every row, HunyuanVideo with the corrected gate). ΔPSNR: paired difference to KalSea at the same budget, mean and 95% bootstrap CI over the videos; win: fraction of videos on which the variant has higher PSNR than KalSea. Last two rows of each block: gain of PCGS on reuse and on linear extrapolation (paired, win: fraction of videos improved). Every variant has the mean NFE of KalSea and computed the same steps as KalSea on all 20 videos (HunyuanVideo) and at least 17 of 20 videos (Wan2.1).

HunyuanVideo	δ=0.2575 (24 NFE)					δ=0.48 (16 NFE)				
Variant	PSNR↑	ΔPSNR (dB)	win	SSIM↑	LPIPS↓	PSNR↑	ΔPSNR (dB)	win	SSIM↑	LPIPS↓
KalSea (Kalman + PCGS, RTS gains)	35.13	0 (ref.)	–	0.957	0.029	23.81	0 (ref.)	–	0.823	0.150
Reuse, no correction (SeaCache)	32.26	−2.87 [−3.77, −2.03]	5%	0.939	0.049	22.29	−1.52 [−2.59, −0.51]	40%	0.794	0.193
PCGS with linear-interp. gains	35.18	+0.05 [−0.65, +0.61]	65%	0.957	0.029	24.01	+0.20 [−0.40, +0.75]	85%	0.829	0.145
Smoothed residuals to filter state only	33.85	−1.28 [−1.83, −0.79]	5%	0.950	0.035	22.71	−1.10 [−1.60, −0.64]	5%	0.799	0.178
Reuse + PCGS	34.60	−0.54 [−1.62, +0.50]	40%	0.953	0.032	23.44	−0.37 [−1.57, +0.89]	45%	0.815	0.161
Linear extrapolation, no correction	33.58	−1.55 [−2.20, −0.91]	5%	0.948	0.037	22.54	−1.27 [−1.85, −0.77]	0%	0.798	0.178
Linear extrapolation + PCGS	35.08	−0.06 [−0.43, +0.28]	45%	0.956	0.030	23.64	−0.17 [−0.69, +0.33]	40%	0.819	0.155
gain of PCGS on reuse		+2.33 [+1.88, +2.77]	100%				+1.15 [+0.57, +1.83]	80%		
gain of PCGS on linear extrapolation		+1.49 [+0.92, +2.15]	100%				+1.10 [+0.58, +1.67]	90%		
Wan2.1	δ=0.2 (24.05 NFE)					δ=0.35 (17 NFE)				
Variant	PSNR↑	ΔPSNR (dB)	win	SSIM↑	LPIPS↓	PSNR↑	ΔPSNR (dB)	win	SSIM↑	LPIPS↓
KalSea (Kalman + PCGS, RTS gains)	30.04	0 (ref.)	–	0.951	0.023	21.12	0 (ref.)	–	0.816	0.119
Reuse, no correction (SeaCache)	28.71	−1.33 [−2.23, −0.47]	35%	0.934	0.038	21.60	+0.49 [−0.26, +1.22]	60%	0.814	0.125
PCGS with linear-interp. gains	29.95	−0.09 [−0.31, +0.17]	30%	0.950	0.023	21.15	+0.04 [−0.06, +0.13]	60%	0.820	0.117
Smoothed residuals to filter state only	29.70	−0.34 [−0.77, +0.07]	40%	0.951	0.024	21.07	−0.04 [−0.22, +0.15]	40%	0.814	0.123
Reuse + PCGS	30.16	+0.11 [−0.66, +0.88]	55%	0.950	0.024	22.16	+1.04 [+0.38, +1.72]	75%	0.838	0.100
Linear extrapolation, no correction	29.65	−0.39 [−0.93, +0.15]	40%	0.951	0.024	21.24	+0.12 [−0.30, +0.54]	55%	0.818	0.119
Linear extrapolation + PCGS	29.73	−0.31 [−0.77, +0.08]	50%	0.949	0.025	21.41	+0.30 [−0.04, +0.67]	60%	0.824	0.110
gain of PCGS on reuse		+1.45 [+1.13, +1.77]	100%				+0.55 [+0.27, +0.86]	75%		
gain of PCGS on linear extrapolation		+0.08 [−0.46, +0.62]	50%				+0.17 [−0.12, +0.44]	75%		

methods are therefore not run-to-run noise; their intervals reflect the variability over prompts only. The twelve videos of Figs. S4 and S5 (reference, SeaCache and KalSea for two prompts per model) were generated again for this supplement with the same code and seeds; their frames are bit-identical to those of the evaluated videos, and their per-video PSNR equals the evaluated value.

H. TFLOPs and latency

TFLOPs. We use SeaCache's convention (Calflops): twice the multiply-accumulates of the linear, convolution and matrix-product operations of the denoiser over the sampling loop; attention cores, text encoders and the VAE are not counted. We count them with PyTorch's FLOP counter on one generation per configuration. Our counts of the 50-step originals reproduce the published ones (FLUX.1-dev: 2975.6 vs. 2976; HunyuanVideo: 14043 vs. 14038), so the baselines' published TFLOPs are used as they are. SeaCache's Wan2.1 TFLOPs correspond to 81 frames (8214 for the original; our count at 65 frames is 6649), so those rows are rescaled by $6649/8214$. PCGS and the filter add less than 0.1% to the count of SeaCache at the same NFE (output layers only; elementwise operations are not counted).

Latency. End-to-end time of one generation call (text encoding, sampling and VAE decoding; synchronized), bf16, one NVIDIA B200 per job with all eight GPUs of the node running measurement jobs, after a warm-up call in the same process. One prompt per configuration: FLUX.1-dev, DrawBench "A red colored car." (seed 0, median of three calls); video, VBench "In a still frame, a stop sign" (seed 42, one call). On these prompts SeaCache and KalSea compute exactly the tables' NFE at both budgets. The original, SeaCache and KalSea run with our code; TeaCache, TaylorSeer and fewer-step sampling with their official implementations at the published settings (TaylorSeer on FLUX.1-dev: order 1, first five steps computed; the single-block calls of TeaCache's FLUX.1-dev forward are adapted to the current diffusers interface, with the same operations). As a check, the 50-step original timed with the baselines' drivers agrees with our script to within 0.1% (FLUX.1-dev, HunyuanVideo) and 1.3% (Wan2.1). Δ-DiT and ToCa were not run. At equal NFE, KalSea takes 3–5% longer than SeaCache on FLUX.1-dev and 0.3–1.4% longer on video, for elementwise work that the count omits (filter updates, gate refresh).

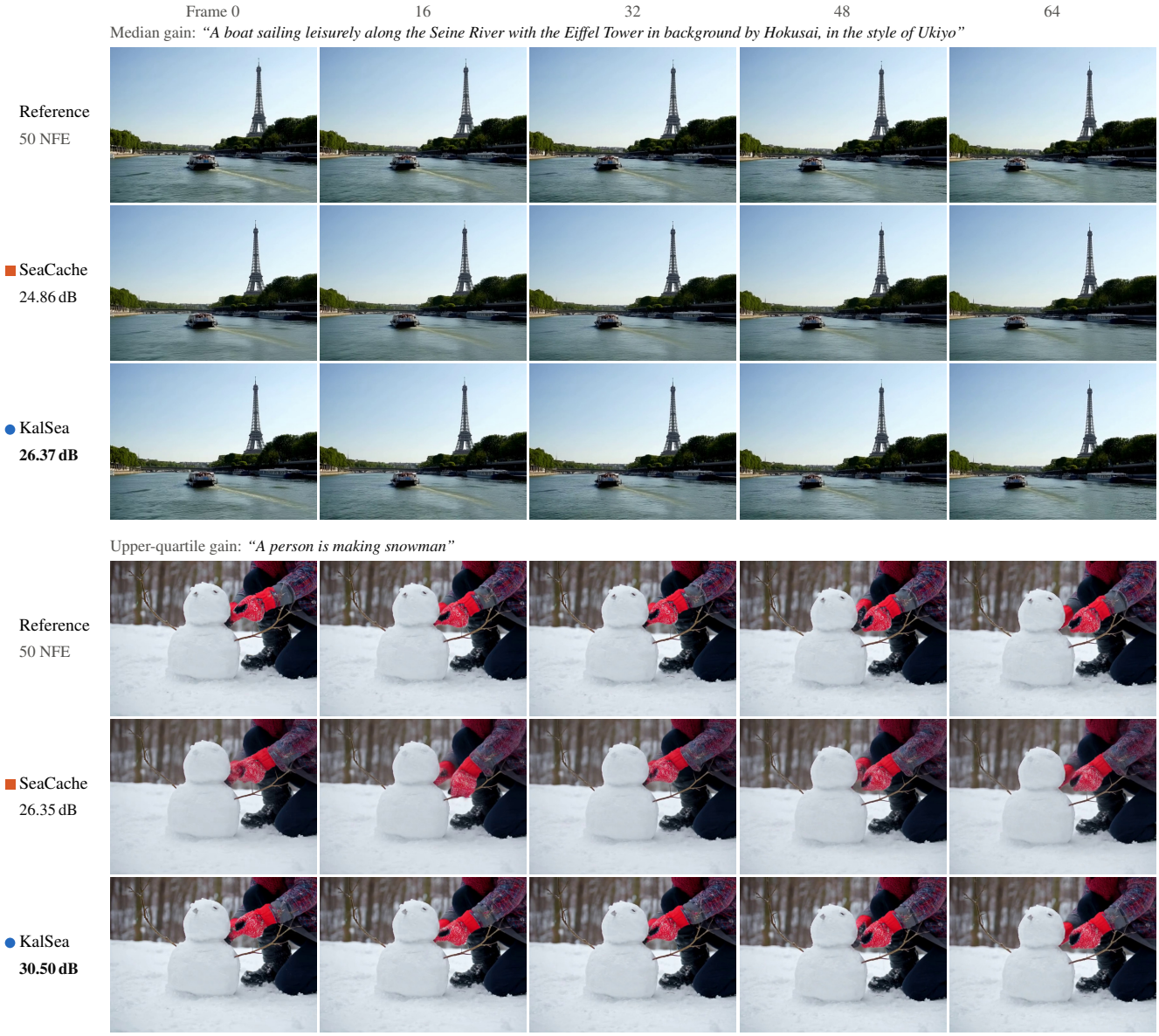


Figure S4. **Video examples, HunyuanVideo** at 16 NFE ($\delta = 0.48$, gate on the sampler's noise level; VBench, 65 frames), frames 0, 16, 32, 48 and 64. Fixed selection rule: among the 936 of 944 prompts on which SeaCache and KalSea computed the identical steps, the videos whose per-video PSNR gain of KalSea over SeaCache is closest to the median (+1.51 dB; top) and to the upper quartile (+4.15 dB; bottom; also in Fig. 4). Row labels: per-video PSNR (mean over the 65 frames) with respect to the 50-step reference, best in bold.

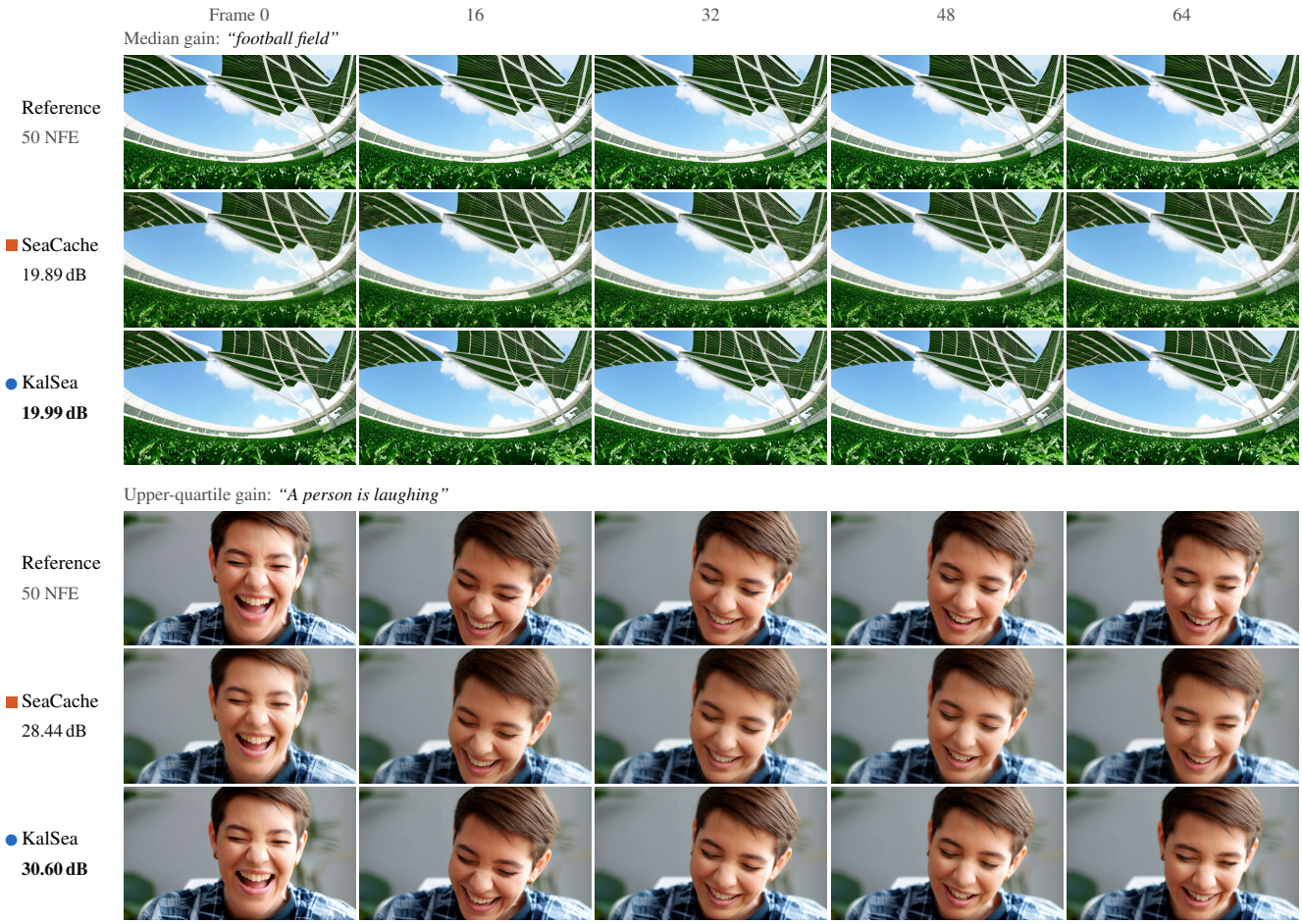


Figure S5. **Video examples, Wan2.1-1.3B** at 17 NFE ($\delta = 0.35$; VBench, 65 frames), frames 0, 16, 32, 48 and 64. Fixed selection rule: among the 729 of 944 prompts on which SeaCache and KalSea computed the identical steps, the videos whose per-video PSNR gain of KalSea over SeaCache is closest to the median (+0.10 dB; top) and to the upper quartile (+2.16 dB; bottom; also in Fig. 4). Row labels: per-video PSNR (mean over the 65 frames) with respect to the 50-step reference, best in bold.

References

[1] Pengtao Chen, Mingzhu Shen, Peng Ye, Jianjian Cao, Chongjun Tu, Christos-Savvas Bouganis, Yiren Zhao, and Tao Chen. Δ-DiT: A training-free acceleration method tailored for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024. 4

[2] Jiwoo Chung, Sangeek Hyun, MinKyu Lee, Byeongju Han, Geonho Cha, Dongyoon Wee, Youngjun Hong, and Jae-Pil Heo. SeaCache: Spectral-evolution-aware cache for accelerating diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14283–14294, 2026. 2, 4, 9, 14

[3] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 10

[4] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21807–21818, 2024. 9

[5] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, et al. HunyuanVideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024. 10

[6] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014. 2, 6

[7] Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7353–7363, 2025. 4, 10, 14

[8] Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Junjie Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15853–15863, 2025. 4, 14

[9] R. Mehra. Approaches to adaptive filtering. *IEEE Transactions on Automatic Control*, 17(5):693–698, 1972. 1

[10] Javad Rajabi, Hakki Can Karaimer, David B. Lindell, Babak Taati, and Michael S. Brown. SpecKCast: Spectral Kalman forecasting for diffusion acceleration. *arXiv preprint arXiv:2610.26837*, 2026. 1

[11] H. E. Rauch, F. Tung, and C. T. Striebel. Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8): 1445–1450, 1965. 3

[12] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013. 3

[13] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. 10

[14] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 586–595, 2018. 4

[15] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 49842–49869, 2023. 10

[16] Chang Zou, Xuyang Liu, Ting Liu, Siteng Huang, and Linfeng Zhang. Accelerating diffusion transformers with token-wise feature caching. In *International Conference on Learning Representations (ICLR)*, 2025. 4